ACS MILESTONE 31

System Reference

A technical map of how ACS fits together, with concrete end-to-end flows for runtime movement, exits/portals, inspection, editor tile painting, entity placement, validation, persistence, and rendering.

Table of Contents

1. Purpose
2. Feature Implementation Catalog
3. End-To-End Runtime Command Pattern
4. End-To-End Editor Mutation Pattern
5. Milestone 24 Starter Libraries And Graphics Authoring
6. Deployment, Publishing, And Artifact Handoffs
7. AI-Agnostic Provider Architecture
8. Testing, Quality Gates, And Documentation Validation
9. UX Skinning Inventory
10. Future Milestone Direction

Companion documents: the user guide explains how to use the application; this system reference explains what happens internally. The Markdown source includes Mermaid diagrams, and this PDF includes readable rendered equivalents plus screenshots.

Latest Milestone 31 foundation update: `@acs/ai-core` now defines AI-provider manifests, provider registries, adventure-generation requests, proposal envelopes, generation plans, proposal review reports, portable session records, proposal change summaries, application plans, portable review packages, export-ready review bundles, archive-ready review handoffs, shared AI handoff integrity reports, AI handoff import plans, AI handoff import reports, and shared request/proposal validation so future LLM adapters can plug into one structured contract.

Feature Implementation Catalog

This section is the detailed implementation map for the current Milestone 31 application. The important architectural rule is that game meaning lives in shared data and pure domain/runtime packages, while browser UI, canvas rendering, publishing handoff metadata, AI-provider contracts, AI review reports, AI session records, AI change summaries, AI application plans, AI review packages, AI review file bundles, AI review archives, AI handoff integrity reports, AI handoff import plans, AI handoff import reports, and documentation screenshots are presentation layers around that data.

The editor should follow the same rule as the runtime renderer. A WorldTree skin, classic skin, or later branded UX shell should still sit over the same draft `AdventurePackage`, validation/reporting flow, project-release services, and `editor-core` mutation helpers. Visual layout can change; authoring behavior should not fork into skin-specific business logic.

Feature	User-facing behavior	Implementation path
Sample adventure loading	The runtime and editor both start from the same Solar Seal sample adventure.	`apps/web/src/sampleAdventure.ts` exports raw content. `packages/content-schema` reads and normalizes it into an `AdventurePackage`. Runtime and editor clone that package instead of maintaining separate content models.
Classic ACS / Debug Grid visual modes	The player can switch presentation without changing game state.	`apps/web/src/index.ts` tracks the selected visual mode. `packages/runtime-2d` renders the same `RuntimeSnapshot` differently. The engine never branches rules by visual style.
Keyboard movement	Arrow/WASD movement updates player position, turn count, events, triggers, and enemies.	The browser converts keys into runtime commands. `GameSession.dispatch` handles movement, trigger execution, enemy turn cadence, and emits events. The browser rerenders from the returned snapshot/events.
Keyboard interaction and inspection	`E` interacts with adjacent entities; `Q` inspects nearby game state.	The browser dispatches `interact` or `inspect`. Runtime-core finds directional targets, emits inspection/interaction events, and runs matching interaction triggers.
Dialogue	Classic mode shows dialogue in the bottom message band; Debug Grid uses the larger dialogue panel.	Dialogue records live in the adventure package. Trigger actions set active dialogue in runtime state. Browser presentation decides whether the dialogue is shown in the classic text band or debug panel.
Trigger actions	Tiles can give items, set flags, show dialogue, play media cues, play sound cues, teleport, change tiles, and advance quest-like state.	Triggers are structured arrays of `Condition` and `Action` objects. `runtime-core` evaluates conditions and applies actions. The editor can author these through guided controls while preserving structured data.
Classic size scaling	Players can resize the Classic ACS canvas to Compact, Large, or Extra Large without restarting or changing game state.	`apps/web/src/index.ts` stores a `classicScale` preference and calls `CanvasGameRenderer.setClassicScale`. `runtime-2d` computes classic metrics from renderer options.
Media and sound cues	Trigger chains can emit named splash/transition/media and sound/ambient/music cue events.	`packages/domain` defines cue objects and `playMedia`/`playSound`. Validation checks cue references. Runtime-core emits `mediaCuePlayed` and `soundCuePlayed`; the browser event log resolves cue names.
Quest definitions and objectives	Designers create/edit quest stages, rewards, and source references; the runtime Objective panel reads current quest state.	`QuestDefinition` records live in domain data. `editor-core` creates/updates quests, `setQuestStage` trigger actions mutate `state.questStages`, validation checks quest references, and `apps/web` summarizes the current objective.
Turn-based enemy behavior	Enemies act after configured successful player turns rather than every blocked step.	Entity behavior profiles are definition data. Runtime-core advances turn counters on successful commands and evaluates behavior intervals, detection radius, leash distance, and wander options.

Runtime save/load	The player can save, load, and reset session state locally.	`packages/persistence` stores `RuntimeSnapshot` records in IndexedDB. The save wraps canonical runtime state; it does not duplicate map definitions or invent a second state model.
Focused editor navigation	The editor shows one coherent workspace at a time: Adventure, World Atlas, Map Workspace, Libraries, Logic, or Test & Publish.	`apps/web/editor.html` marks sections with `data-editor-areas`. `apps/web/src/editor.ts` tracks `activeEditorArea`, applies hash/query startup state, and toggles visibility.
Map creation and metadata	Designers can create maps, assign regions, and classify map kind.	`editor-core` creates cloned map definitions with tile layers. The browser form updates `MapDefinition` fields and validation checks references.
Tile brush	A selected tile behaves like a paint brush and can be dragged across many cells.	The browser tracks brush state across pointer events and calls `setTileAt`. The grid cell refreshes without requiring a full mental reset after each tile.
Entity instances	Designers can move existing entities or place new instances from reusable definitions.	Definitions live in `entityDefinitions`. Instances store `definitionId`, optional `displayName`, optional `behaviorOverride`, `mapId`, and coordinates. Placement policy allows singleton or multiple-instance definitions.
Classified libraries	Items, dialogue, flags, quests, skills, traits, spells, assets, and custom object classes are browsed by library focus.	The domain model includes reusable objects and `LibraryCategoryDefinition`. The editor filters summaries and category creation by focus so objects are not just typed strings in unrelated fields.
Stocked starter genre libraries	Milestone 24 exposes real starter objects for fantasy, sci-fi, modern spy, superhero, science-fantasy, supernatural, and urban-fantasy adventure building.	`sampleAdventure.ts` defines genre-specific tile, entity, item, skill, trait, spell, and asset objects. `StarterLibraryPackDefinition` groups those ids without changing runtime rules.
Pixel-art and presentation authoring	Designers can choose splash/music/intro text and paint classic 8x8 pixel sprites from the Assets library focus.	`packages/domain` defines presentation, starter-pack, and pixel-sprite records. `packages/editor-core/src/asset-authoring.ts` clones and updates presentation/pixel data. `apps/web/src/editor.ts` renders the Assets focus and pixel grid.
Exits and portals	Designers can connect maps by selecting a target map/coordinate and clicking a source cell.	Milestone 20 adds editor-core exit helpers, browser Exits & Portals layer mode, dependency summaries, and runtime movement through `MapDefinition.exits`.
Tile definition library	Designers can create/edit reusable terrain definitions with passability, hints, tags, categories, and classic sprite mappings.	Milestone 21 adds `TileDefinition` records, editor-core helpers, Libraries/Tiles controls, validation of tile references, runtime terrain blocking, and runtime-2d sprite-id resolution through tile definitions.
Project API and releases	Drafts can be validated, saved as projects, published, and opened as releases.	`apps/api` stores project/release data in `apps/api/data/store.json`. Browser project controls call `packages/project-api`, while validation is shared with the local editor.
AI-agnostic provider contracts	Future LLM or agent integrations can target one shared request/proposal contract without directly mutating the editor or runtime.	`packages/ai-core` defines `AiProviderManifest`, `AdventureGenerationRequest`, `AiAdventureProposal`, provider registries, generation plans, proposal review reports, session records, proposal change summaries, application plans, portable review packages, export-ready review bundles, archive-ready review handoffs, shared AI handoff integrity reports, AI handoff import plans, AI handoff import reports, and shared request/proposal validation helpers.

Quality gate	New or changed code should remain easier to understand.	`tools/complexity-check.mjs` rejects new/worsened functions above cyclomatic complexity 8. The baseline records legacy violations to reduce over time.
Full testing gate	Milestones must prove behavior at package, editor UI, runtime UI, command-runtime, complexity, and documentation-validation levels before completion.	`tools/run-unit-tests.mjs` runs Node tests against compiled packages. `tools/editor-ui-smoke.ps1` drives the real browser editor in headless Chromium. `tools/runtime-ui-e2e.ps1` drives the real playable browser runtime in headless Chromium. `tools/playtest-smoke.mjs` dispatches runtime commands through `createGameEngine`.

Milestone 31 starts by creating a pure provider-contract seam instead of wiring any one model SDK directly into the editor. `@acs/ai-core` now owns provider manifests, generation requests, proposal envelopes, provider-registry helpers, generation plans, proposal review reports, session records, proposal change summaries, application plans, portable review packages, export-ready review bundles, archive-ready review handoffs, and request/proposal validation. This keeps future AI-assisted authoring reviewable and structured before any mutation reaches the adventure package.

Milestone 31F adds the first portable AI review handoff object: `AiGenerationSessionPackage`. It bundles the existing session record, proposal impact summary, application plan, required file manifest, recommended package names, and README guidance into one export-ready structure so later UI and API layers can persist or hand off a single reviewed artifact.

Milestone 31G then turns that package into concrete export-ready files through `AiGenerationSessionPackageFileBundle`, including `package-manifest.json` plus the session, change-summary, application-plan, and README payloads. That means later export code can trust one declared bundle format instead of inventing separate serialization rules.

Milestone 31H adds `AiGenerationSessionPackageArchive` so those export-ready files can also be emitted as one archive artifact with declared entries and archive bytes. That keeps archive packaging inside the same pure AI handoff layer instead of forcing later UI or API code to rebuild ZIP behavior ad hoc.

Milestone 31I adds `AiGenerationSessionHandoffReport` so the package, file-bundle, and archive layers can now be judged together from one shared integrity/readiness object. Later editor or API surfaces can use that report directly instead of reconstructing AI handoff export/import readiness in presentation code.

Milestone 31J adds `AiGenerationSessionImportPlan` so a later review, audit, or import surface can decide whether a reviewed AI handoff is safe to ingest, which artifacts it preserves, and what still blocks import, without rebuilding those rules in UI code.

Milestone 31K adds `AiGenerationSessionImportReport` so that same later review, audit, or import surface can present import readiness in one reviewer-facing summary: import status, blocker counts, required file coverage, preserved artifacts, summary lines, and one next-step recommendation from the reviewed handoff state.

The intended long-term AI flow is deliberately staged so no model provider ever gets a direct line into mutable game data.

1. A caller creates an `AdventureGenerationRequest`.
2. `createAdventureGenerationPlan(...)` produces the human-review-first workflow for that request.
3. A provider adapter sends the normalized request to one specific model API.
4. The adapter converts the provider response into one `AiAdventureProposal`.
5. `createProposalReviewReport(...)` evaluates request validity, proposal validity, provider warnings, and review readiness.
6. `createGenerationSessionRecord(...)` bundles the request, plan, proposal, and review report into one stable session object.
7. `createProposalChangeSummary(...)` explains what the proposal would change.
8. `createProposalApplicationPlan(...)` answers whether the proposal is safe to apply and which content sections it would touch.
9. `createGenerationSessionPackage(...)` and `createGenerationSessionPackageFileBundle(...)` turn that reviewed state into a portable handoff package for persistence, export, or later import.
10. Only after explicit human acceptance should normal editor mutation helpers apply the structured changes to the live adventure draft.

That separation matters. The model provider is only responsible for returning structured content or patch intent. The review, apply, export, and persistence rules stay inside ACS-owned code.

The current codebase does not yet include concrete OpenAI, Anthropic, or local-model adapters. The provider-agnostic boundary was built first so those adapters all have to follow the same path.

1. Declare one `AiProviderManifest`.
2. Register that manifest inside an `AiProviderRegistry`.
3. Accept a normalized `AdventureGenerationRequest`.
4. Convert the request into the provider's transport shape.
5. Ask the provider for structured output.
6. Convert the provider response back into one `AiAdventureProposal`.
7. Preserve provenance: `providerId`, optional `providerLabel`, optional `model`, and `generatedAt`.
8. Return the proposal for normal ACS review helpers to validate and package.

The important design rule is that the adapter is a translation layer, not a business-rules layer.

Once provider adapters exist, switching models should be operationally simple because the rest of the review flow stays the same.

1. Add or update an `AiProviderManifest` with fields such as `id`, `displayName`, `transport`, `capabilities`, `supportsStructuredOutput`, and `modelHints`.
2. Point the adapter at the desired provider endpoint or SDK.
3. Map the same `AdventureGenerationRequest` into that provider's prompt, tool, or schema format.
4. Map the provider response back into the shared `AiAdventureProposal` shape.
5. Run the normal ACS review helpers: `createProposalReviewReport(...)`, `createGenerationSessionRecord(...)`, `createProposalChangeSummary(...)`, `createProposalApplicationPlan(...)`, `createGenerationSessionPackage(...)`, and `createGenerationSessionPackageFileBundle(...)`.
6. Compare the reviewed outputs, not just the raw model text.

Switching models should not require rewriting editor workflows, persistence, runtime logic, or publishing logic. It should mostly require a different provider manifest, a different adapter, and possibly different prompt or schema tuning.

1. Create a new adapter module outside `packages/ai-core` so `@acs/ai-core` stays pure and network-free.
2. Define the provider manifest in that adapter.
3. Implement request translation: `AdventureGenerationRequest -> provider request payload`.
4. Implement response translation: `provider response payload -> AiAdventureProposal`.
5. Add tests for manifest registration, request translation, response translation, and invalid provider response handling.
6. Plug the adapter into whichever future surface calls AI: local API, future editor AI panel, or batch generation tool.
7. Keep the rest of the lifecycle unchanged: request -> plan -> proposal -> review report -> session -> change summary -> application plan -> package -> apply only after acceptance.

That is the main benefit of the AI-agnostic model layer. The provider can change without changing the authoring and review contract.

ACS currently has two human-facing release handoff modes plus one reviewer package. They all start from the same rule:

1. Edit a draft.
2. Validate it.

3. Publish an immutable release.
4. Export handoff artifacts from that release.

The release is the source of truth for deployment. ACS does not deploy directly from a mutable draft.

Use this when another designer should be able to continue development.

1. Publish a release.
2. Export the forkable package from `Test & Publish`.
3. The package ZIP contains editable handoff files such as `forkable-project.json`, `project-manifest.json`, README files, release notes, and shared handoff metadata.
4. A future import flow will use that package as the seed for a new mutable project.

Use it to share with another designer, archive a remixable release, or later import it into ACS as a new project instead of treating it as a live shared workspace.

Use this when players should only play the game, not edit it.

1. Publish a release.
2. Export the standalone package from `Test & Publish`.
3. ACS builds a release-backed static web bundle.
4. The ZIP contains the runtime shell plus packaged content such as `index.html`, `styles.css`, runtime scripts, `bundle/adventure-package.json`, `bundle/distribution-manifest.json`, README files, release notes, and optional local-launch helpers.

Use it by hosting the bundle as a static website or by using the packaged launcher helpers for local play. Later desktop or mobile wrappers can still sit on top of the same bundle.

Use this when someone needs to review the release package itself rather than play or edit it.

1. Publish a release.
2. Preview release handoff and artifact integrity inside `Test & Publish`.
3. Export the review package.
4. The review ZIP contains `RELEASE-HANDOFF.json`, `ARTIFACT-INTEGRITY.json`, README files, release notes, and review-package manifest files.

Use it for QA review, release sign-off, or archival validation of what exactly was shipped.

The important architectural split is simple: `forkableProject` is for future editing, `standalonePlayable` is for playing, and the review package is for auditing the release handoff itself. All three come from the same immutable release, which keeps release identity, release notes, package naming, packaged manifests, and validation expectations consistent across the whole deployment story.

If the goal is to collaborate with another designer: validate draft, publish release, preview the forkable artifact, then export the forkable package.

If the goal is to ship a game to players: validate draft, publish release, preview the standalone package, check release readiness, export the standalone ZIP, then host or launch the packaged bundle.

If the goal is to review a release package: validate draft, publish release, preview release handoff, preview artifact integrity, preview the review package, then export the review package.

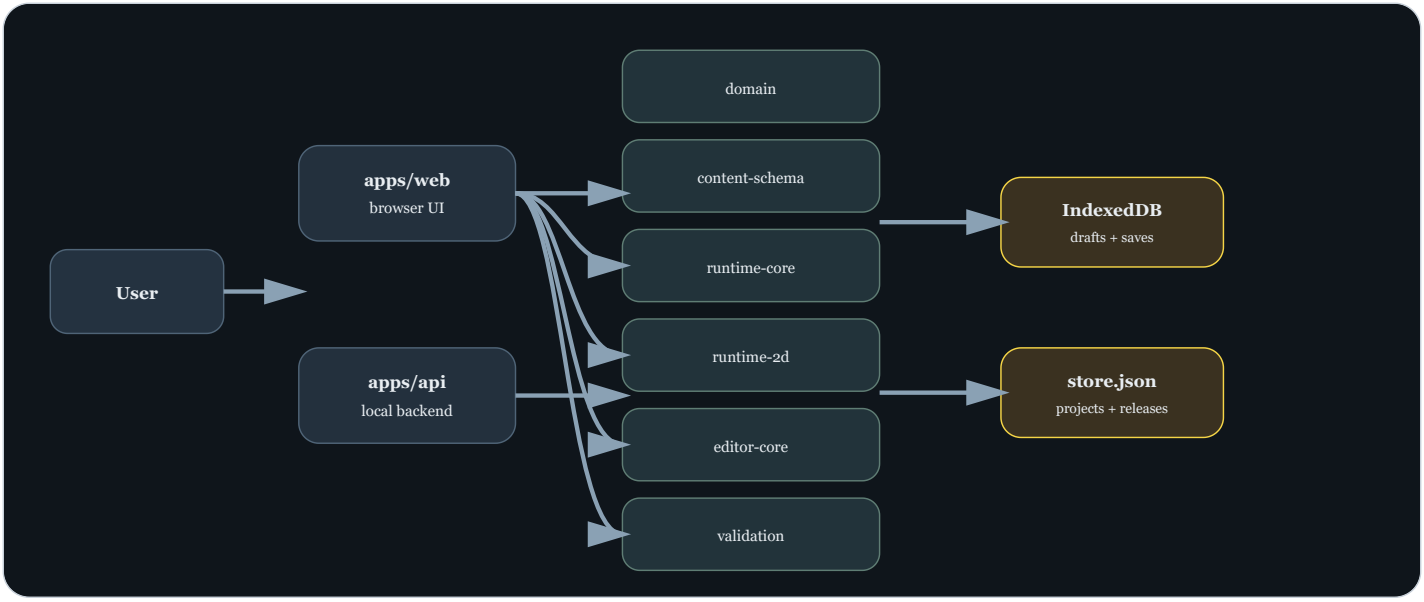
All player commands follow the same core loop: browser input becomes a typed runtime command, runtime-core mutates canonical session state, runtime-core emits events, the browser app records user-facing messages, and runtime-2d renders from a snapshot.

```
Keyboard or UI input
  -> apps/web command handler
  -> GameSession.dispatch(command)
  -> runtime-core state mutation and EngineEvent[]
  -> apps/web event log/message band/state panels
  -> runtime-2d canvas draw from RuntimeSnapshot
```

Editor operations follow a parallel loop: a focused panel collects user intent, browser code builds a small input object, editor-core clones and updates the adventure package, validation reruns, and the current workspace rerenders from the updated draft.

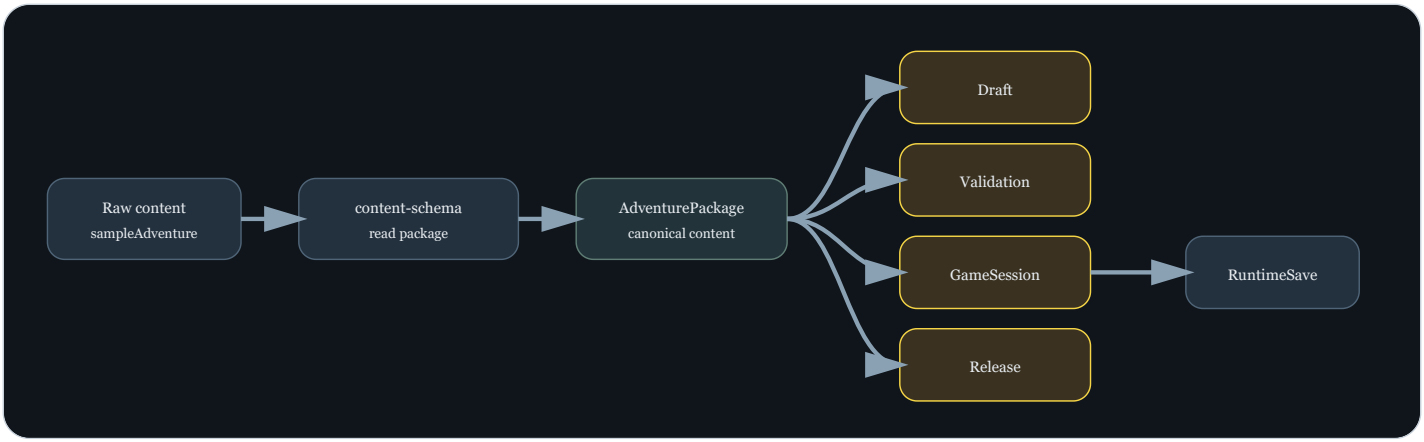
```
Focused editor control
  -> apps/web/src/editor.ts reads form/grid input
  -> editor-core pure helper clones and updates AdventurePackage
  -> packages/validation checks references and content rules
  -> editor rerenders grid, summaries, validation, and project controls
```

High-Level Architecture



Area	Package or app	Responsibility
Shared vocabulary	`packages/domain`	IDs, maps, entities, triggers, dialogue, actions, and conditions.
Content ingestion	`packages/content-schema`	Reads raw authored content and normalizes it into an `AdventurePackage`.
Simulation	`packages/runtime-core`	Public exports live in `index.ts`; command orchestration lives in `game-session.ts`; triggers, enemy turns, state hydration, and movement math live in focused runtime modules.
Rendering	`packages/runtime-2d`	Draws runtime state to canvas without owning game rules.
Editing	`packages/editor-core`	Pure draft operations such as `setTileAt`, `addEntityInstance`, and `moveEntityInstance`.
Validation	`packages/validation`	Publish-readiness checks shared by browser and API.
Persistence	`packages/persistence`	IndexedDB runtime saves and local editor drafts.
Backend	`apps/api`	Projects, releases, validation endpoint, and file-backed storage.

Data Model Layers



Key distinction: a release is immutable content, while a save is runtime state. A save wraps the engine's existing `RuntimeSnapshot`, so persistence does not invent a second save-game model.

Current Screens

EDIT FLOW

1. Adventure
2. World Atlas
3. Map Workspace
4. Libraries
5. Logic
6. Test & Publish

MAP WORKSPACE

Terrain, Entities & Local Objects

Work inside the selected map. Terrain paints the tile layer; entity mode moves or places instances that reference reusable definitions.

Map: Sun Meadow
Layer Mode: Terrain Tiles
Tile Brush: Sun Grass (grass, passable)
Move Instance: entity_oracle (Oracle)
Place Definition: Shrine Wolf (multiple)
Target Map: Sun Meadow
Target X: 0
Target Y: 0
Delete Selected Exit

ACTIVE TOOL
Sun Grass (grass, passable)

grass	grass	shrub	grass	grass	grass	grass	grass
grass	path	path	path	grass	grass	grass	grass
grass	path	grass	ORAC	grass	grass	grass	grass
grass	path	grass	path	grass	grass	shrub	grass
grass	path	grass	path	path	WOLF	path	grass
grass	path	grass	grass	grass	grass	path	grass
grass	path	path	path	path	path	door	grass
grass	grass	grass	grass	grass	grass	grass	grass

Painting tiles on Sun Meadow. Brush tile: grass. Click and drag to paint multiple cells.

DEPENDENCIES

Selected Map Relations

Use this as the mental checklist for the selected map: it belongs to a region, contains entities, may have exits, and can be referenced by triggers.

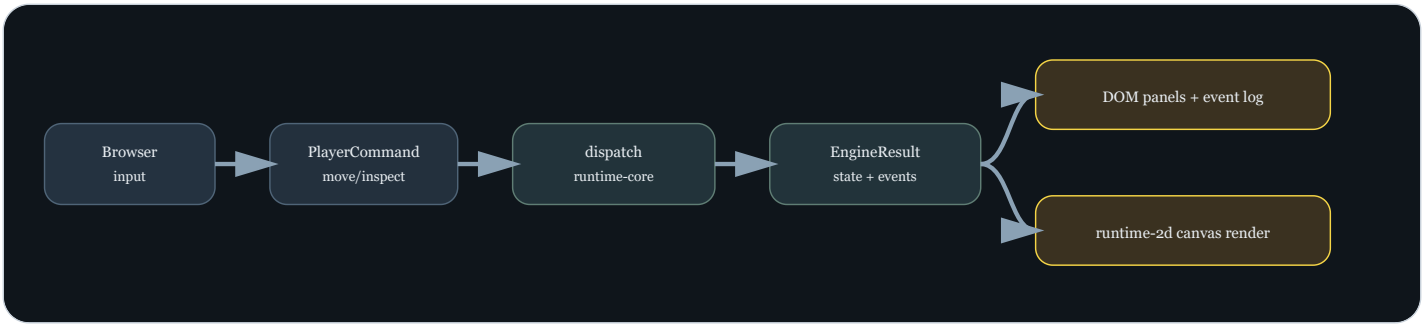
REGION
Map metadata

TERRAIN
Tile layer

POPULATION
Entity instances

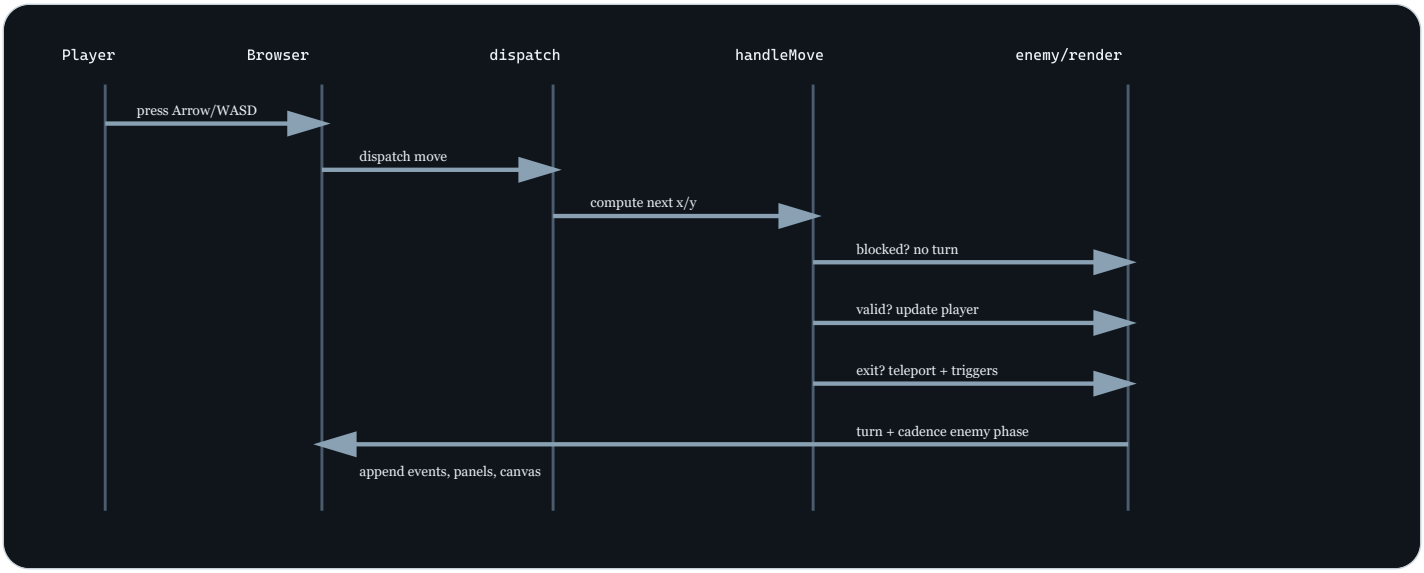
LOGIC
Triggers & exits

Runtime Input To Rendering



Runtime input follows a narrow path: `apps/web/src/index.ts` turns browser input into a `PlayerCommand`, `runtime-core` mutates session state and emits events, then the browser updates panels and asks `runtime-2d` to redraw.

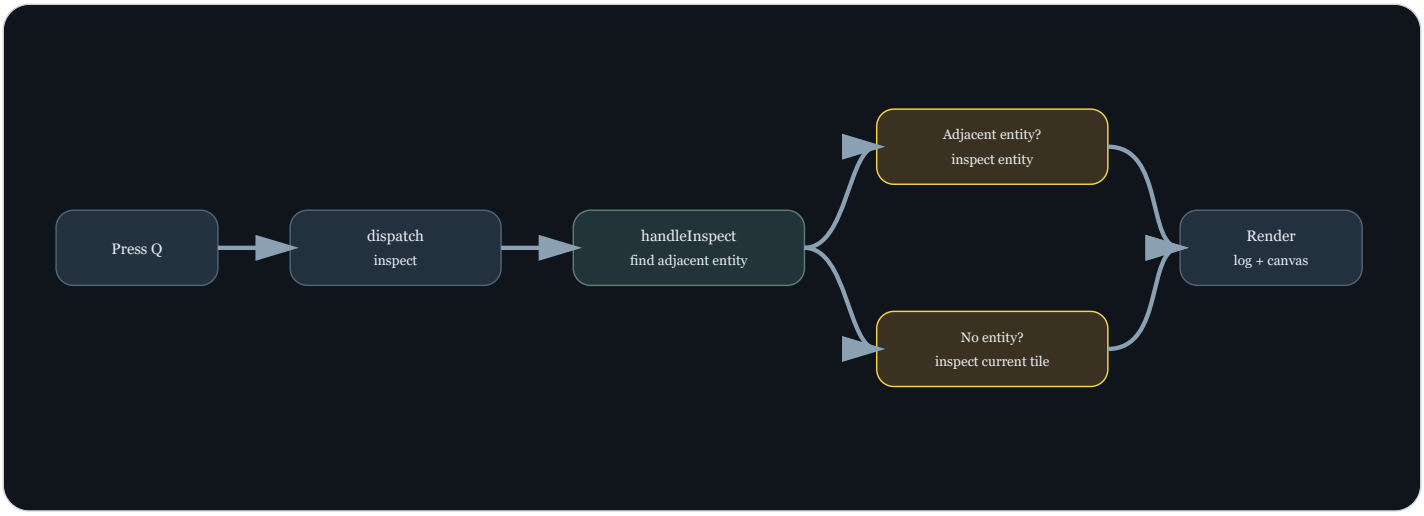
Use Case 1: Move Command



1. The keydown handler in `apps/web/src/index.ts` maps Arrow keys and `WASD` to `{ type: "move", direction }`.
2. `runCommand` calls `session.dispatch(...)`, appends returned events, and calls `renderEverything`.
3. `dispatch` calls `handleMove` and uses the returned boolean to decide whether the command consumed a turn.
4. `handleMove` computes the next coordinate, rejects bounds or occupied destinations, or updates `state.player`.
5. If the destination is an exit, the engine changes `currentMapId`, teleports the player, and runs map-load and tile triggers.
6. If movement succeeded, `dispatch` increments `state.turn`, emits `turnEnded`, and runs the enemy phase.
7. The browser updates map name, player position, flags, inventory, dialogue, event log, and canvas.

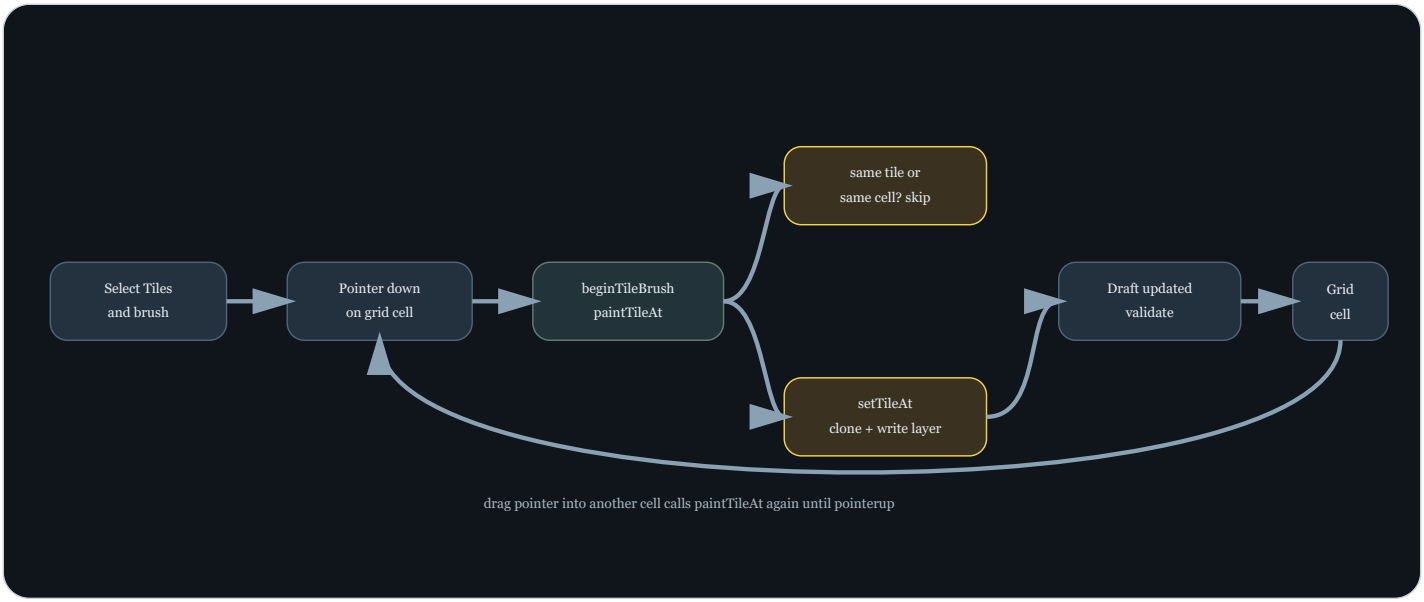
Current behavior: blocked movement does not consume a turn and does not grant enemies a free action. Enemy behavior profiles may define `turnInterval`; the sample Shrine Wolf uses `turnInterval: 3`, so it only acts on every third successful player turn.

Use Case 2: Inspect Command



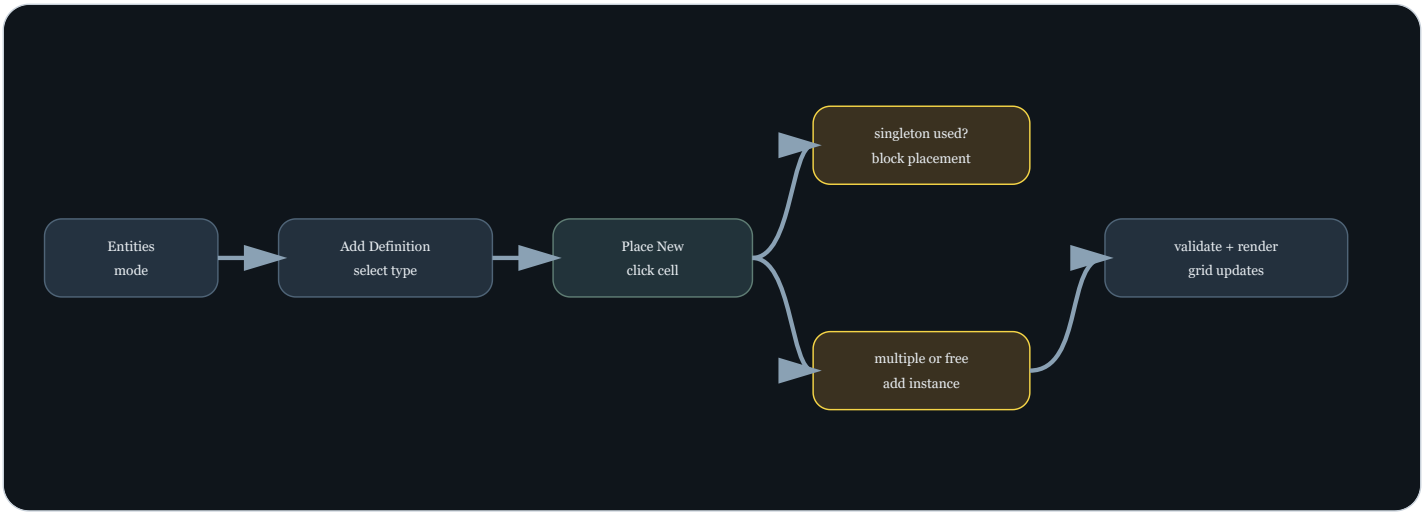
1. The browser maps `Q` to `{ type: "inspect" }`.
2. `dispatch` calls `handleInspect(undefined, events)` and treats a completed inspection as a turn-consuming action.
3. With no explicit direction, `findEntityInDirection` searches for the first active adjacent entity on the current map.
4. If an adjacent entity exists, the engine emits `inspectResult` for that entity.
5. If no adjacent entity exists, the engine emits an `inspectResult` describing the player's current coordinate and map id.
6. Inspect does not directly move the player, grant items, set flags, or start dialogue.
7. `dispatch` increments `state.turn`, emits `turnEnded`, then runs the enemy phase. Enemies still respect their configured `turnInterval`.

Use Case 3: Editor Tile Brush



1. The designer selects `Tiles` mode and picks a tile id. The browser stores it in `selectedTileId`.
2. `renderBrushPreview` updates the visible active brush.
3. Each grid cell has a `pointerdown` handler in tile mode. Pressing a cell calls `beginTileBrush(x, y)`.
4. `beginTileBrush` sets `isTileBrushActive = true`, clears the duplicate-cell guard, and calls `paintTileAt`.
5. `paintTileAt` skips if the editor is not in tile mode, if the same cell was already painted in this drag, or if the cell already has the selected tile.
6. If the tile differs, the browser calls `setTileAt(draft, currentMapId, x, y, tileId)` from `editor-core`.
7. `setTileAt` clones the package, finds the map and active layer, calculates `index = y * map.width + x`, writes the tile id, and returns a new draft.
8. The browser refreshes the changed grid cell, reruns local validation, and updates project/publish controls.
9. Dragging into another cell calls `paintTileAt` through `pointerenter`. Pointer up ends the brush.

Use Case 4: Editor Entity Placement



1. The designer switches to `Entities` mode.
2. The editor shows `Move Entity`, `Add Definition`, `Place New`, `Instance Name`, and `Behavior Override`.
3. Choosing a reusable definition stores `selectedEntityDefinitionId` and enters new-instance placement mode.
4. `singleton` definitions are blocked after one instance exists anywhere in the adventure.
5. `multiple` definitions can be placed repeatedly.
6. `addEntityInstance` clones the package, creates a unique id, applies optional `displayName` and `behaviorOverride`, pushes an `EntityInstance`, then validation and grid rendering rerun.

Entity Definition Vs Instance

An `EntityDefinition` is the reusable template. It answers, "What kind of thing is this?" Examples include `Spy Handler`, `Police Officer`, `Defense Drone`, and `Oracle`. The definition owns reusable behavior defaults, profile data, placement policy, visuals, category, and starting possessions.

An `EntityInstance` is the placed copy. It answers, "Which one is this, and where is it?" Instances carry map position plus optional `displayName` and `behaviorOverride`. That lets a designer place the same reusable definition more than once while naming the copies `Red Team Spy Handler` and `Green Team Spy Handler`, or using a `guard` override on one copy while another inherits the definition behavior.

Runtime behavior resolution first finds the entity definition, then checks the authored instance. If `EntityInstance.behaviorOverride` exists, the enemy turn system uses it. Otherwise it falls back to `EntityDefinition.behavior` and normalizes that value into an `EntityBehaviorProfile`.

Current note: instance creation and movement now support friendly instance names and simple per-instance behavior overrides. Browser UI deletion of instances and creation of brand-new entity definitions remain future work.

Milestone 27 Editor Information Architecture

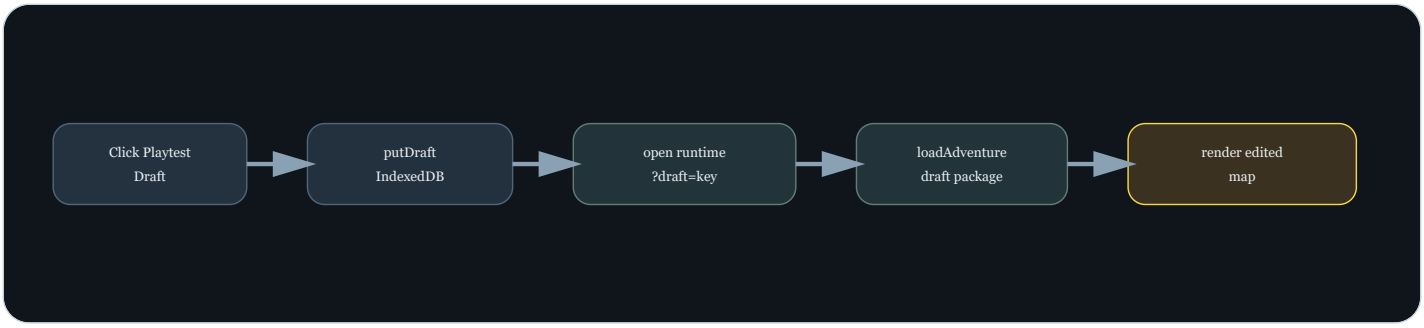
Milestone 27 adds two practical information-architecture tools: a Selected Cell Inspector in Map Workspace and a Display Rename / Reskin workflow in Test & Publish.

The Selected Cell Inspector derives its answer from the current `AdventurePackage` draft. When a designer clicks a map cell, the browser stores the selected coordinate and reads the tile layer, placed entity instance, exit records, and trigger records for that same coordinate. The panel then summarizes tile name/passability, occupant, exit destination, and local triggers without creating a second state model.

The Display Rename / Reskin tool calls shared `editor-core` helpers. `previewDisplayRename` returns before/after matches for player-facing text fields. `applyDisplayRename` clones the draft and updates display fields only. Stable ids, trigger references, map ids, item ids, quest ids, and entity definition ids are preserved.

This is the safe path for future reskins: designers can rename visible concepts from the UI without scripts and without accidentally breaking structured references or save data.

Editor To Playtest



Painting a tile changes only the editor draft. To see it in play, the editor saves the draft to IndexedDB and opens the runtime with a draft query parameter. The runtime loads that same `AdventurePackage`, so there is no separate editor-only map format.

Validation, Publishing, Save, And Load

Validation and publishing

1. Local edits rerun `validateAdventure(draft)`.
2. `Validate Draft` calls the API validation endpoint.
3. Project save and publish are disabled when local validation is blocking.
4. The API validates again before creating an immutable release.

Runtime save/load

1. `Save` calls `session.serializeSnapshot()`.
2. `persistence.saveSession` stores the snapshot in IndexedDB.
3. `Load` retrieves the record and calls `engine.loadAdventure(activeAdventure, snapshot)`.
4. The browser rerenders from restored state.

EDIT FLOW

1. Adventure
2. World Atlas
3. Map Workspace
4. Libraries
5. Logic
6. Test & Publish

WORLD ATLAS

Regions & Maps

Move from region-level organization into the map you want to edit. Maps belong to regions; exits and triggers then attach to maps.

Adventure Package

- Regions
- Maps
- Tiles / Entities / Triggers

Selected Map

Sun Meadow

Current Map Name

Sun Meadow

Map Category

Local

Region

Sun Meadow

Sun Meadow: 8x8, local, region Sun Meadow.

CREATE MAP

New Map Name

Moonlit Road

Category

Local

Region

No region

Width

8

Height

8

Fill Tile

grass

Create Blank Map

DEPENDENCIES

Selected Map Relations

Use this as the mental checklist for the selected map: it belongs to a region, contains entities, may have exits, and can be referenced by triggers.

REGION	TERRAIN
Map metadata	Tile layer
POPULATION	LOGIC
Entity instances	Triggers & exits

ENTITIES ON SELECTED MAP

- entity_oracle: Oracle at (3, 2)
- entity_wolf: Shrine Wolf at (5, 4)

EXITS ON SELECTED MAP

- exit_meadow_to_shrine: (6, 6) -> Inner Shrine (1, 4)

MAP GRAPH

- Sun Meadow (6, 6) -> Inner Shrine (1, 4)

Future Publishing Modes

Publishing should support two different author goals without forcing them into the same artifact. A designer may want to share an editable construction-set project that another creator can fork, or they may want to ship a finished adventure as a standalone playable game with no editor surface.

Mode	Purpose	Contents	Should Exclude
`forkableProject`	Let another designer inspect, remix, and continue authoring the game.	Editable adventure source, authoring metadata, custom libraries, starter-library provenance, dependency graph, license/remix metadata.	Nothing required for responsible editing; source should remain visible.
`standalonePlayable`	Let players run the finished adventure without seeing the construction tools.	Validated immutable release, runtime assets, runtime shell, save/load support, distribution metadata.	Editor UI, draft-only metadata, unused library objects, authoring diagnostics, private construction notes.

Recommended package

A future `packages/publishing` module should own export logic instead of putting it in the editor or runtime. Core functions should include `createForkableProjectExport`, `createStandaloneRuntimeExport`, `collectRuntimeAssets`, `pruneUnusedAuthoringData`, and `validatePublishArtifact`.

Release path

The safe flow is: mutable project draft, validation, immutable published release, then one or more export artifacts. Exporting must not mutate the draft or release. It should only transform a known-good release into a shareable package.

Distribution metadata

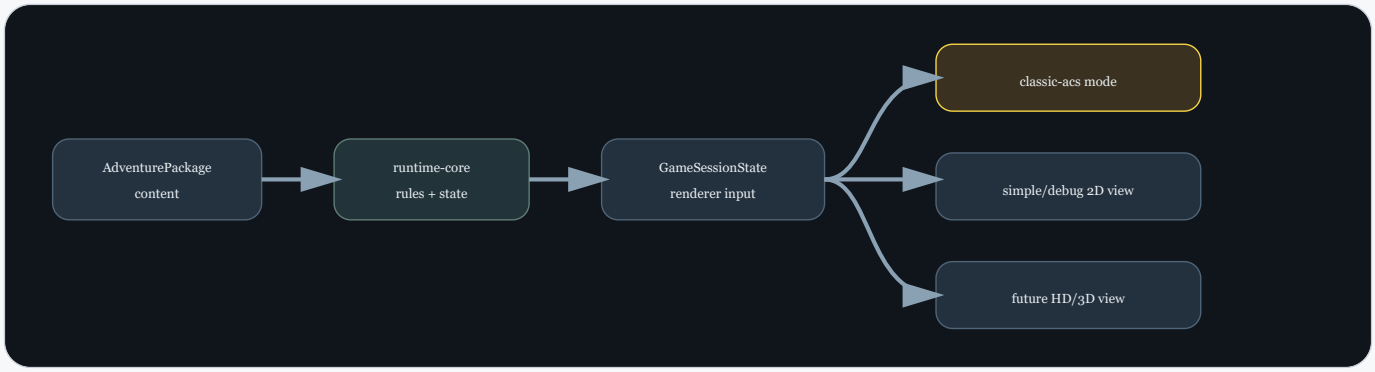
Every exported artifact should state `artifactType`, `editable`, `sourceIncluded`, `allowForking`, `license`, and `runtimeOnly`. That makes the file self-describing for humans, the app, and future AI tooling.

Forward Milestone Path: Classic ACS Feel

The project should deliberately capture the feel of the original 1980s Adventure Construction Set while keeping the engine free of renderer assumptions. The legacy image set points to two major tracks: a classic gameplay panel and deeper construction-set authoring tools.

Classic runtime presentation

The classic runtime mode should be a presentation layer over the same `AdventurePackage`, `RuntimeSnapshot`, and `GameSessionState` used by the current renderer.



- fixed-aspect vintage gameplay panel
- tile/icon map viewport on a dark field
- right-side status rail with clearly labeled life, power, and future actor-resource meters
- bottom message band for location names, prompts, scrollable Classic ACS dialogue, interaction text, and command hints
- pixelated sprite scaling and controlled palette
- asset IDs resolved through manifests so the same map can later render with HD or 3D assets

Classic editor capability track

The original editor suggests future modes for terrain pictures, thing pictures, creature pictures, map/floor wiring, reusable definitions, actor profiles, possessions, triggers, portals, text, and dialogue.

1. **Milestone 10:** completed `classic-acs` visual mode with a gameplay viewport, right status rail, bottom message band, and renderer theme switching.
2. **Milestone 11:** completed sprite manifests, a first classic asset set, entity sprite IDs, manifest-driven classic renderer lookup, and validation warnings for missing sprite references.
3. **Milestone 12:** completed entity definition editing for reusable metadata, placement policy, sprite asset ids, faction, and behavior tuning.
4. **Milestone 13:** completed dialogue text editing and structured trigger editing for existing records.
5. **Milestone 14:** completed map categories, current-map structure metadata editing, and blank map creation.
6. **Milestone 15:** completed richer entity profiles, skills, starting possessions, runtime party/profile/inventory rendering, editor fields, and validation checks.
7. **Milestone 16:** completed no-code trigger/action construction for existing trigger records, including guided condition/action add/remove controls and an advanced JSON mirror.
8. **Milestone 17:** completed trigger creation, duplication, deletion, map-cell trigger markers, and trigger reference summaries.
9. **Milestone 18:** focused editor workspaces for Adventure, World Atlas, Map Workspace, Libraries, Logic, and Test & Publish.
10. **Milestone 19:** map context selectors and classified libraries.
11. **Milestone 20:** exits, portals, and map graph tools.
12. **Milestone 21:** tile definition library with passability, tags, interaction hints, and renderer-neutral visual bindings.
13. **Milestone 23:** quest and objective builder.
14. **Milestone 23:** object-model corrective pass with quest-local objectives, reusable objective templates/archetypes, anti-duplicate library organization, TypeScript section/module cleanup, then creature interaction foundations, combat, drops, and defeat triggers.
15. **Milestone 24:** classic pixel-art, splash, music, and stocked genre library authoring, including a true built-in pixel editor, reusable fantasy/science-fiction/modern-spy/superhero/science-fantasy/supernatural-investigation/urban-fantasy starter libraries, adventure splash-screen selection, starting music selection, visual manifest editing, and future HD 2D pack preparation.
16. **Milestone 25:** authoring diagnostics and playtest harness.
17. **Milestone 26:** import/export and schema migration hooks.
18. **Milestone 27:** editor information architecture completion around the Proposed Editor Areas, including a UI-based game-wide display rename/reskin tool with preview, scoped replacement, undo-safe draft edits, validation after apply, and a clear separation between safe display-name changes and advanced internal ID refactors.
19. **Milestone 28:** Classic ACS presentation polish and accessibility scaling.
20. **Milestone 29:** validation, testing, content QA, and documentation acceptance checks.
21. **Milestone 30:** MVP packaging with an Adventuria-inspired sample adventure using the stocked starter libraries across fantasy, sci-fi, modern/spy, superhero, science-fantasy, supernatural investigation, urban fantasy, and castle-style scenes.

Genre Library Policy

The stocked starter libraries should be genre-complete enough to let a designer begin making a game immediately. The default library should include original, generic assets and definitions inspired by broad genres, not direct copies of protected intellectual property.

For example, the library can support superhero adventures with capes, secret identities, gadgets, power icons, city rooftops, and masked villains, but it should not ship Marvel or DC characters, logos, names, or distinctive protected designs. The same principle applies to science-fantasy, supernatural investigation, urban fantasy, modern spy, and other popular genres.

The same caution now applies to product naming. A future legal-safe rebrand milestone should move human-facing ACS / Adventure Construction Set naming toward WorldTree branding. That rename should be handled as a compatibility migration, not a blind search-and-replace, so saved drafts, releases, exports, package names, and storage keys continue to work during the transition.

Current User Guide tutorial requirement

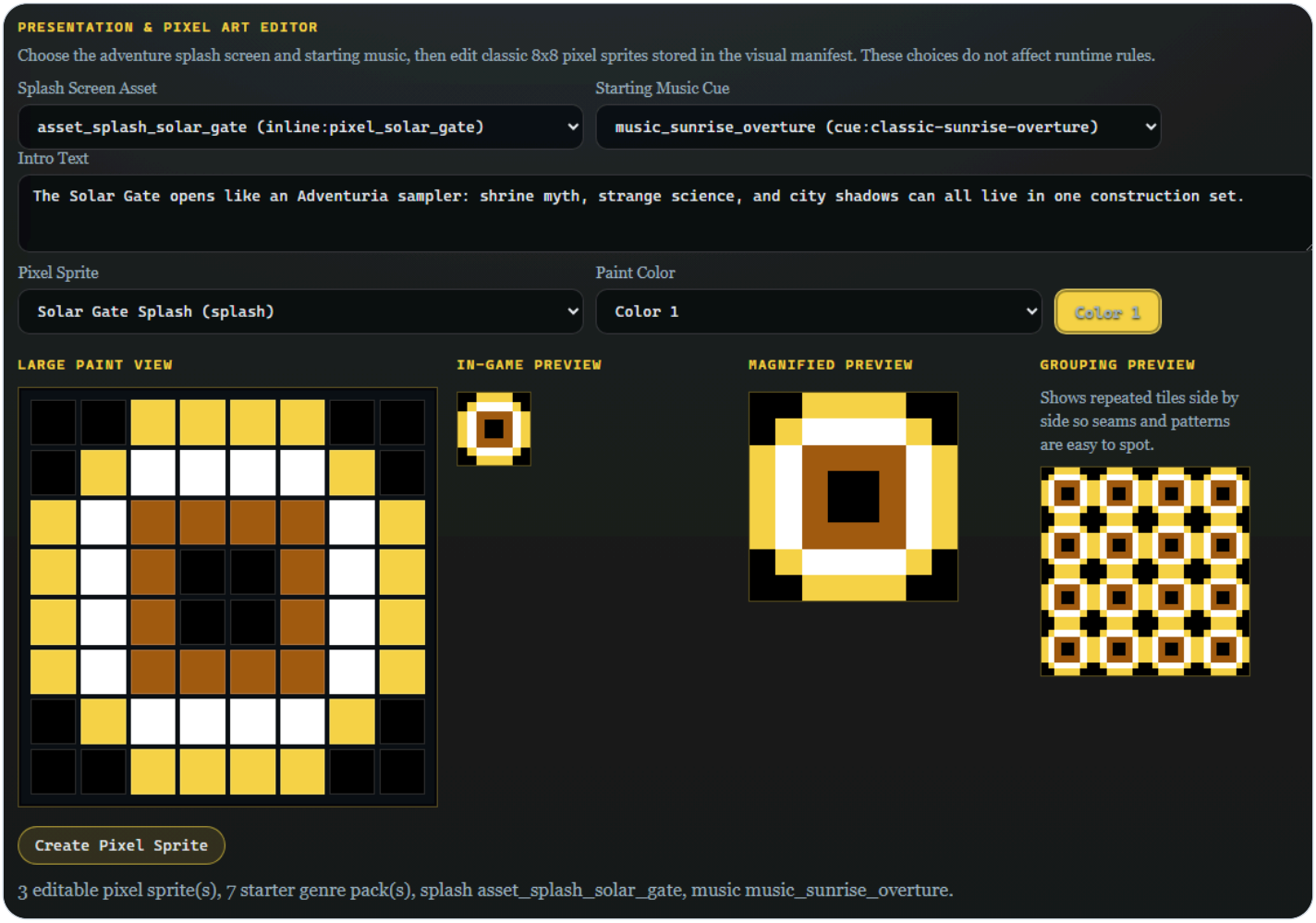
The User Guide tutorial should build a science-fiction adventure from scratch using the strongest currently supported trigger chain. The current tutorial concept is Relay Station Alecto: station maps, terminal/transit/core tiles, a station AI, a defense drone, a multi-stage relay restoration quest, dialogue/flag/item/teleport/tile-change/quest-stage triggers, presentation assets, diagnostics, and playtesting. The example should showcase current framework power without pretending future unsupported effects already exist.

Each genre pack should include

- Terrain and architectural tiles
- Interactable objects and treasure/items
- NPC/enemy/entity definitions
- Portraits or symbolic character icons
- UI/item icons
- Splash-screen templates
- Starting music cues and sound-effect cue ids
- Tags, categories, and example trigger-friendly object definitions

Milestone 24 Presentation, Pixel Art, And Starter Libraries

Milestone 24 adds the first implementation slice for classic ACS-style asset authoring and stocked starter libraries. It introduces adventure-level presentation settings, manifest-backed classic pixel sprites, and genre packs made from real reusable objects. Runtime rules, movement, triggers, and quest state remain independent from presentation data.



The pixel editor renders a visual paint-color swatch plus three synchronized previews from the same sprite record: an in-game preview at normal sprite proportions, a magnified preview for precise painting, and a grouping preview that repeats the sprite in a 4-by-4 tile block. The grouping preview is deliberately modeled after the original ACS graphic editor workflow because repeated terrain, walls, shrubs, floors, and starfields often reveal problems at the seam rather than inside a single isolated tile.

Layer	Object or function	Role
Domain	`AdventurePresentationDefinition`	Stores selected splash asset, starting music asset, and intro text.
Domain	`ClassicPixelSpriteDefinition`	Stores editable classic pixel art as dimensions, palette, pixels, usage, tags, and genre tags.
Domain	`StarterLibraryPackDefinition`	Groups reusable tiles, entities, items, skills, traits, spells, assets, and quests into genre-oriented starter packs.
Content schema	`normalizeVisualManifests`	Ensures pixel sprites have valid dimensions, palettes, and pixel arrays.
Editor core	`asset-authoring.ts`	Contains presentation updates, pixel sprite creation, pixel painting, and starter pack listing.
Browser editor	`Libraries -> Assets`	Shows splash/music selectors, intro text, an 8x8 pixel editor, and starter pack summaries only when Assets is active.
Browser runtime	`Adventure Intro`	Displays selected splash/music ids and intro text without changing engine rules.

```
Designer chooses Libraries -> Assets
  -> editor updates AdventurePackage.presentation
  -> pixel grid updates VisualManifestDefinition.pixelSprites
  -> editor rerenders the in-game, magnified, and grouping previews
  -> draft is saved/playtested
  -> runtime reads presentation data into Adventure Intro panel
```

The sample now includes seven stocked starter packs: Fantasy Shrine Trial, Science Fiction Data Core, Modern Spy Operation, Superhero Rooftop Crisis, Science-Fantasy Gate, Supernatural Case File, and Urban Fantasy Alley. These packs are curated references to existing objects, not separate engines or hardcoded genre modes. Future New Adventure tooling should let designers select one or more genre collections at project creation while still allowing cross-genre borrowing from any collection.

Milestone 28 Classic Presentation, Media Cues, And Sound Cues

Milestone 28 completes another slice of the presentation-is-not-rules architecture. The engine still knows nothing about pixels, audio devices, DOM overlays, or video playback. It only knows that a trigger action requested a media cue or sound cue, and it emits a typed event. The browser presentation layer decides how visible or audible that cue should become.

`AdventurePackage` now includes `mediaCues` and `soundCues`. Each cue points at an `AssetDefinition` by `assetId`. Validation checks that referenced assets exist and warns if cue asset kinds do not match the expected visual or audible role.

```
flowchart LR
    Trigger["TriggerDefinition"] --> Action["Action: playMedia / playSound"]
    Action --> Cue["MediaCueDefinition or SoundCueDefinition"]
    Cue --> Asset["AssetDefinition"]
    Action --> Runtime["runtime-core EngineEvent"]
    Runtime --> Browser["apps/web event log / future playback"]
```

When the player interacts with the Oracle, runtime-core finds the matching trigger, emits `triggerFired`, applies `playMedia` and `playSound`, starts dialogue, sets flags, advances quest state, and returns the updated state plus event list to the browser.

```
interact command
  -> TriggerSystem
  -> mediaCuePlayed(cueId)
  -> soundCuePlayed(cueId)
  -> dialogueStarted + flagSet + questStageSet
  -> browser event log + renderer refresh
```

The no-code trigger builder exposes `Play Media Cue` and `Play Sound Cue` in `Then Actions`. Choosing either action reveals only the relevant cue selector, preserving progressive disclosure.

Classic size is renderer-local. It is stored in browser local storage and passed to `CanvasGameRenderer`. Maps, coordinates, tiles, entities, saves, releases, and trigger rules are unchanged.

Milestone 26 Starter And Custom Library Portability

Milestone 26 creates the first explicit package boundary for starter/custom library storage. The project now has `packages/default-content`, published internally as `@acs/default-content`.

Layer	Owner	Storage Intent
Built-in starter libraries	Application/default content	Shipped curated packs such as science-fiction, fantasy, modern spy, classic ACS reference, and future stocked genre collections.
Adventure-local objects	Project/adventure file	Objects created or copied for one specific game. These can safely diverge from built-in defaults.
Custom library exports	Designer-authored library file	Reusable packs a designer can import into another adventure.

Default content API

- `builtInStarterLibrarySource` identifies application-owned starter content.
- `createLibraryCollections(pkg)` copies reusable object collections from an adventure package.
- `createStarterLibrarySnapshot(pkg)` wraps starter packs and reusable object collections as a built-in snapshot.
- `createCustomLibraryExport(options)` creates designer-owned custom library export records with schema version, genre tags, provenance, and object collections.
- `listStarterPacksByGenre` and `listCustomLibraryObjectCounts` support future selection and import/export UI.

The current implementation establishes the boundary and type-safe file shape first. The next UI layer can add New Adventure genre selection, import/export file pickers, duplicate warnings, and object-level copy/fork controls without changing runtime-core.

Milestone 23 Object-Backed Quest Objectives And Rewards

Milestone 23 continues the object-model corrective pass by changing quest stage text and reward notes from loose string arrays into nested objects. A quest remains the parent library object, while each objective has a stable id, title, kind, description, optional target map, optional target item, and completion-stage number. Each reward has a stable id, kind, label, optional item reference, and optional quantity.

Layer	Object or function	Role
Domain	`QuestDefinition.objectives`	Stores objective records instead of loose stage strings.
Domain	`QuestRewardDefinition`	Stores reward records instead of freeform reward notes.
Content schema	`normalizeQuestDefinitions`	Migrates legacy `stages: string[]` and string rewards into objective/reward objects when raw content is read.
Editor core	`quest-definitions.ts`	Holds quest-specific list/create/update/sanitize helpers outside the all-purpose `index.ts`.
Browser editor	`Libraries -> Quests`	Lets designers select quest objects, objective objects, and reward objects without editing JSON.
Runtime core	`TriggerSystem.setQuestStage`	Still mutates `state.questStages[questId]`, so existing trigger-stage behavior remains stable.
Browser runtime	`summarizeCurrentObjective`	Reads the current objective object by quest stage and displays title, description, and reward labels.
Validation	`questObjectiveCount`	Validates stage numbers against objective count while retaining legacy stage fallback during migration.

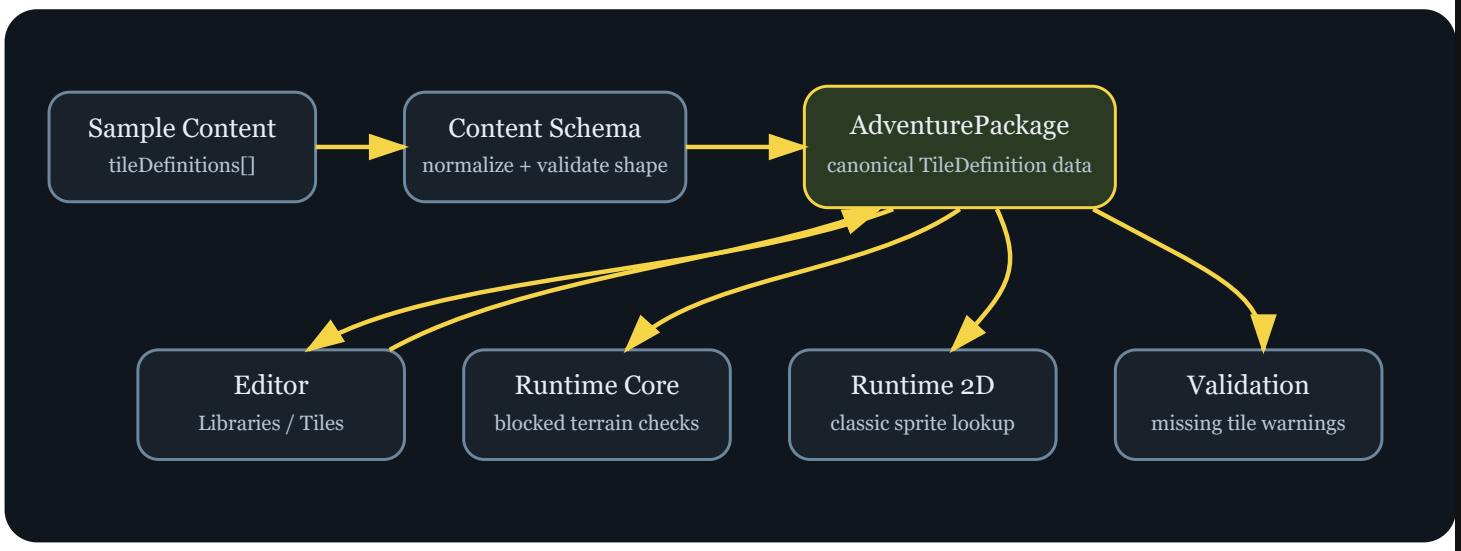
```
Player presses E near Oracle
  -> apps/web dispatches interact
  -> runtime-core matches trigger_intro
  -> TriggerSystem applies setQuestStage(quest_solar_seal, 1)
  -> state.questStages.quest_solar_seal = 1
  -> apps/web summarizeCurrentObjective reads quest.objectives[1]
  -> Objective panel shows objective title, description, and reward labels
```

This keeps quest progression out of page copy and out of renderer code, while giving later milestones a clean place to add objective templates, duplicate warnings, richer completion rules, and reusable reward/effect objects.

Milestone 21 Tile Definition Library

Milestone 21 turns terrain from loose tile ids into reusable library data. A painted map cell still stores a tile id such as `grass`, `water`, `shrub`, or `altar`, but that id now resolves to a `TileDefinition` in the adventure package. This gives the engine, editor, validator, and renderer a shared object to reason about.

Layer	Object or function	Role
Domain	`TileDefinition`	Stable data record with id, name, description, passability, interaction hint, tags, classic sprite id, and optional category id.
Content schema	`normalizeTileDefinitions`	Reads raw tile definitions, supplies safe defaults, and preserves older packages with no tile library.
Editor core	`listTileDefinitions`, `createTileDefinition`, `updateTileDefinition`	Pure draft operations that clone the package before adding or changing reusable terrain records.
Browser editor	`Libraries -> Tiles`	Lets designers browse tile definitions, edit behavior fields, create a new terrain definition, and use the same tile id in the map brush.
Runtime core	`isBlockedTerrain`	Looks up the destination cell's tile definition during movement and blocks movement when passability is blocked.
Runtime 2D	`resolveClassicTileSprite`	Maps logical tile ids to classic sprite ids without forcing future HD or 3D renderers to use the same artwork.
Validation	`validateTileReference`	Warns when map layers or trigger tile-change actions reference tile ids that have no definition.



When the player moves, runtime-core now reads the destination tile id from map layers and tile overrides, resolves that id through `AdventurePackage.tileDefinitions`, and emits `movementBlocked` with reason `terrain` when the definition is blocked. Blocked terrain does not consume a turn, so enemies do not receive a free action.

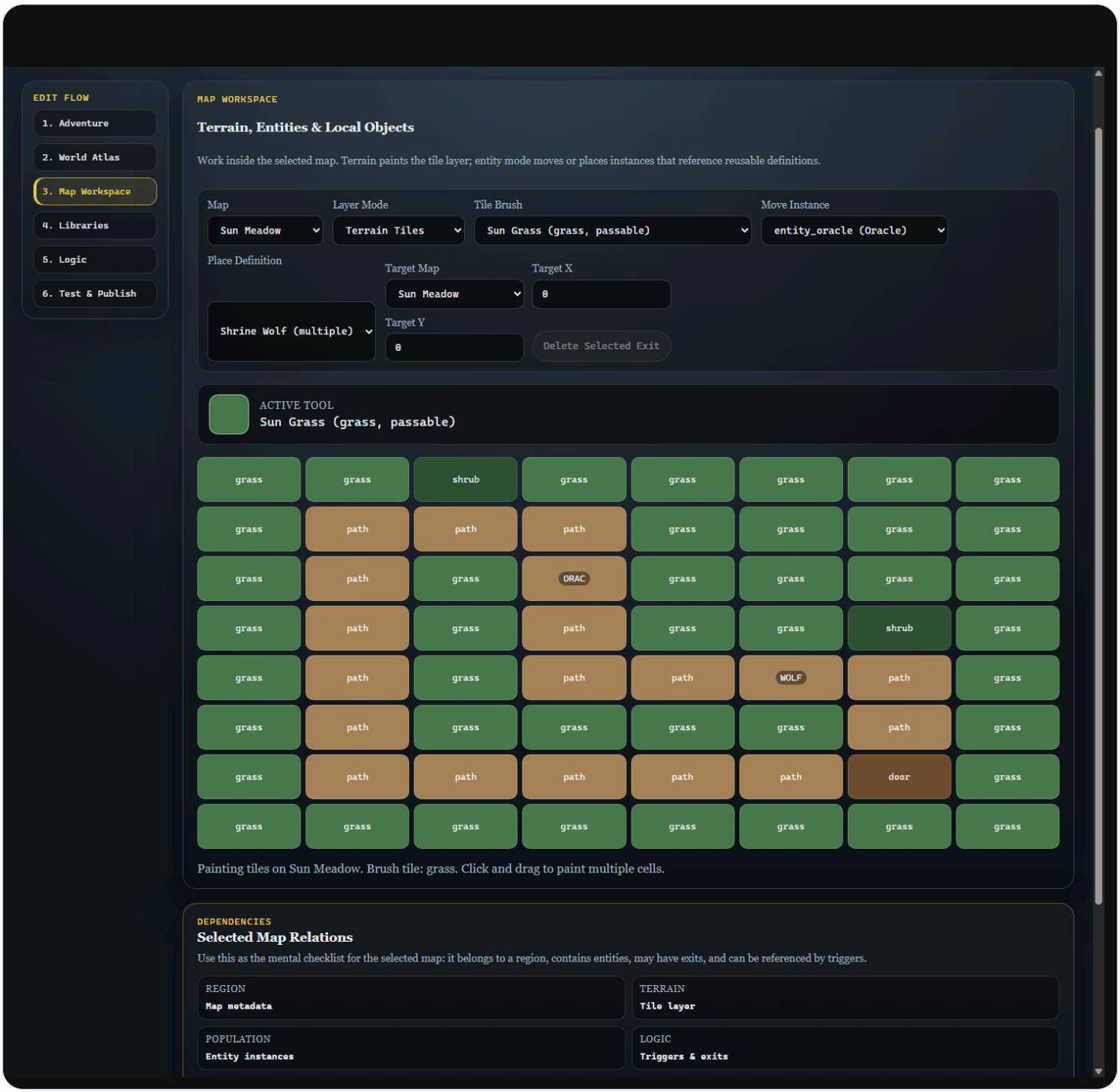
1. The designer opens `Libraries`, chooses `Tiles`, and creates `Force Field`.
2. Browser code calls `editor-core.createTileDefinition`, which clones the draft and appends a sanitized `TileDefinition`.

3. The designer sets passability to `blocked`, adds an interaction hint, tags it `barrier` and `sci-fi`, and assigns a temporary classic sprite.
4. Validation reruns and the tile brush palette now includes the new tile by name.
5. When the designer paints that tile and playtests, runtime-core blocks movement into that cell because it reads the same definition data the editor authored.

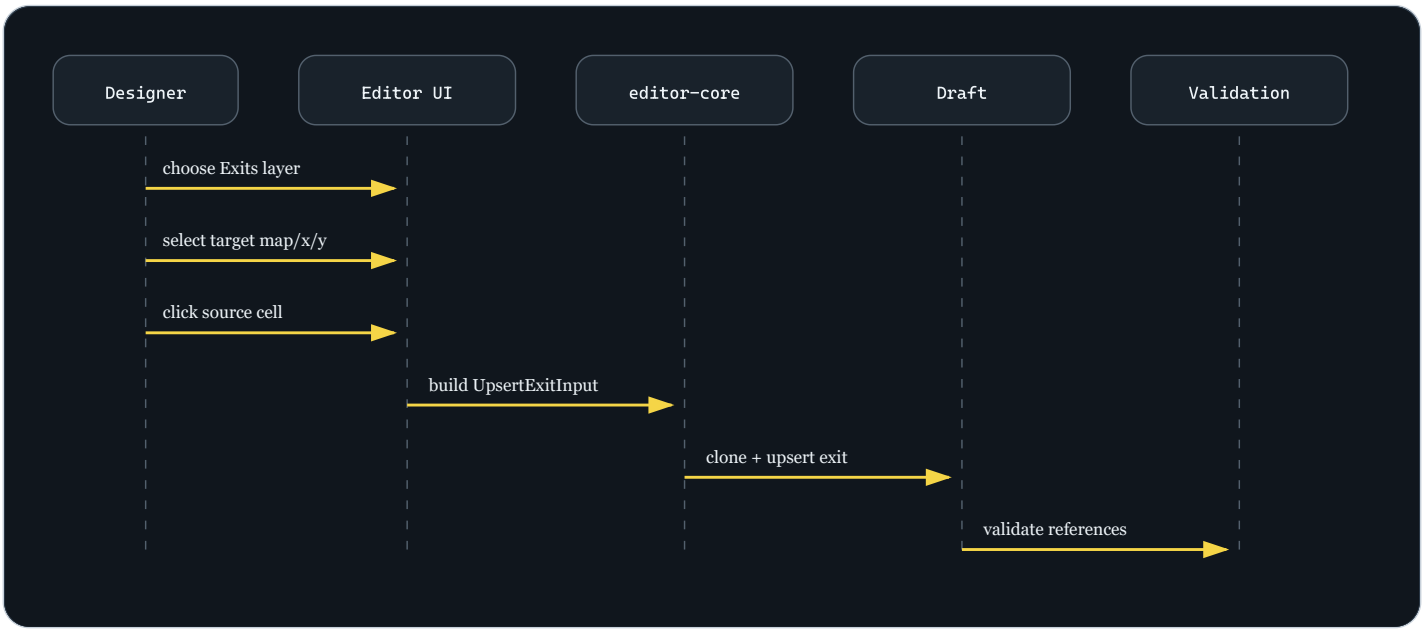
This keeps terrain behavior out of browser UI, keeps visuals out of runtime-core, and prepares the project for future visual modes where the same logical tile can be an 8-bit block, a high-resolution sprite, or a 3D object.

Milestone 20 Exits, Portals, And Map Graphs

Milestone 20 turns travel between maps into first-class authored data. A map owns zero or more `ExitDefinition` records. Each exit has a source coordinate on that map plus a target map and target coordinate. The same data powers editor summaries, validation checks, and runtime teleportation.

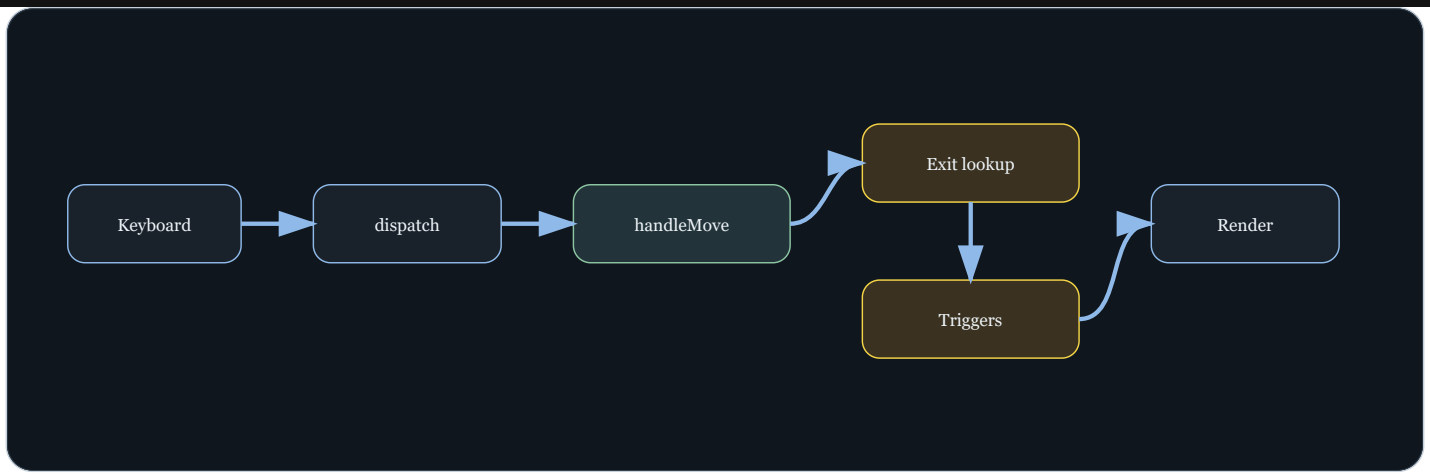


Layer	Object or function	Role
Domain	`ExitDefinition`	Stable data record with `id`, `x`, `y`, `toMapId`, `toX`, and `toY`.
Editor core	`listExitsForMap`, `upsertExitDefinition`, `deleteExitDefinition`	Pure draft operations that clone the package before modifying map exit arrays.
Browser editor	`Exits & Portals` layer mode	Lets the designer choose target map/coordinate, click a grid cell, and inspect or update exits.
Runtime core	`handleMove`	After a successful move, checks whether the destination cell has an exit and transfers the player.
Validation	`validateExits`	Reports broken target maps, out-of-bounds source/target coordinates, and suspicious overlaps.




```
dispatch({ type: "move", direction: "south" })
  -> handleMove(direction, events)
  -> compute nextX/nextY
  -> reject bounds or occupied cells
  -> update player position and emit playerMoved
  -> find currentMap.exits at the new coordinate
  -> if found: set currentMapId, player.x, player.y
  -> emit teleported
  -> run map-load triggers for the target map
  -> run tile triggers at the arrival coordinate
  -> render the new snapshot
```

This means exits are not a renderer trick. The player first moves onto the source cell. Runtime-core then changes the canonical session state to the destination map and coordinate. After that, runtime-2d simply renders whatever the new snapshot says.



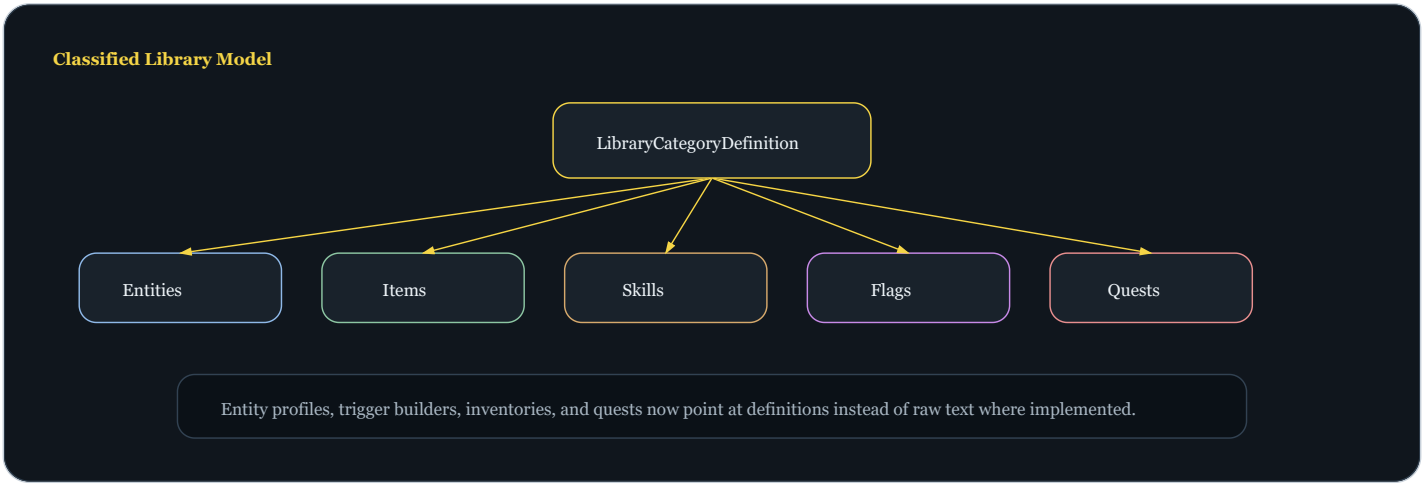
The map graph is intentionally derived, not stored. The editor builds rows by walking `draft.maps.flatMap(map => map.exits)` and rendering each source map to target map edge. That design avoids a second graph model that could drift out of sync with the real exit records.

Validation has the same source of truth. It can verify that every exit points to an existing map, that source and target coordinates are inside their maps, and that authors can notice risky patterns such as multiple exits on the same source cell.

The milestone also applies the cleanup policy going forward: changed functions must stay at cyclomatic complexity 8 or lower, SOLID-style separation must not regress, and extracted helpers should make code easier to hold in your head. During this milestone, the browser grid/edit flow was split into smaller functions such as `wireGridCell`, `wireTileBrushCell`, `renderExitBrushPreview`, `editorHintForMode`, `selectExistingExit`, and `buildExitInput`. The complexity baseline now tracks 23 legacy violations to pay down over time after the runtime refactor and Milestone 21 editor cleanups, and the Milestone 21 changes add no new complexity violations.

Milestone 19 Map Context And Classified Libraries

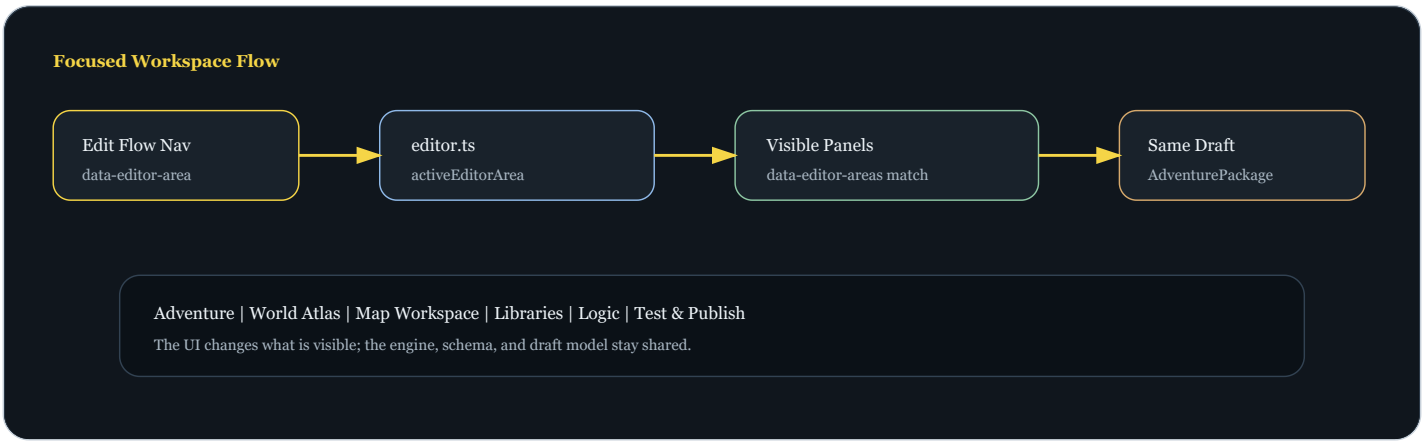
Milestone 19 extends the focused workspace idea downward into the data model. The editor can now keep map context close to map work, and the content schema can classify reusable objects instead of leaving future concepts as typed strings.



- `packages/domain` defines library categories, skills, traits, spells, flags, and custom library objects.
- `packages/content-schema` normalizes missing arrays and migrates legacy profile skills into skill ids.
- `packages/validation` checks category references and entity skill/trait references.
- `apps/web/editor.html` adds map selectors inside Map Workspace and Logic, plus a Library Focus selector for entities, items, skills, traits, spells, dialogue, flags, quests, tiles, assets, and custom objects. The Tiles focus now exposes concrete tile definition editing for passability, hints, tags, and classic sprite bindings.

Milestone 18 Focused Editor Workspaces

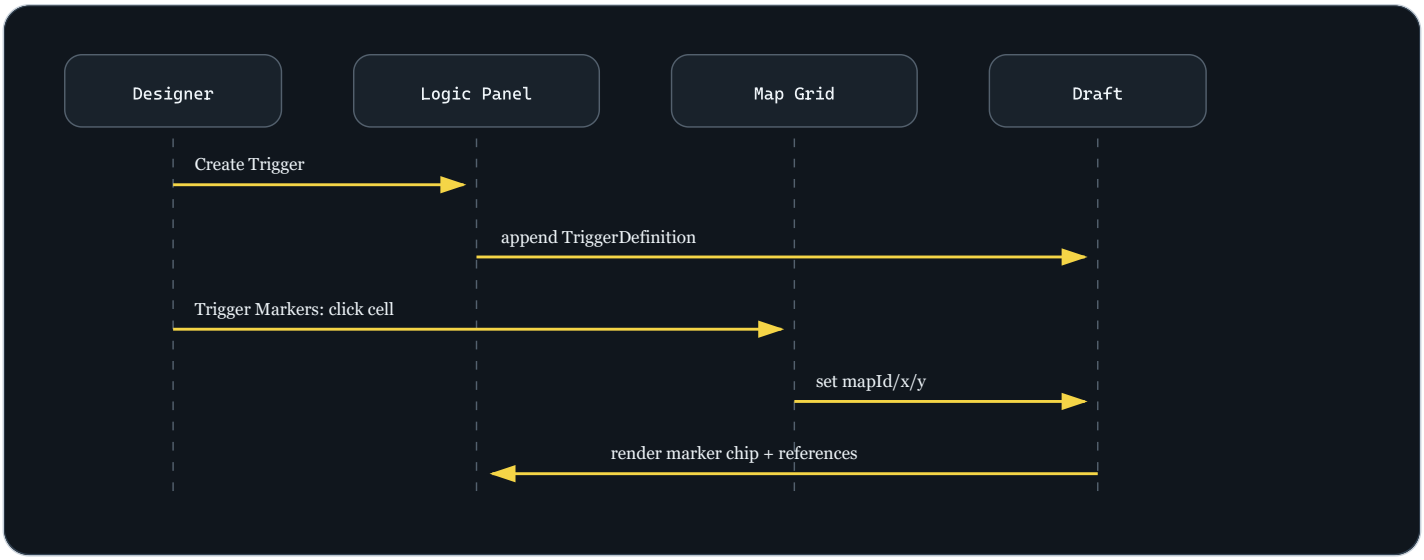
Milestone 18 changes the editor from a long all-panels dashboard into a focused workspace switcher. The browser page still uses the same editor-core operations and AdventurePackage draft, but the visible UI is filtered by the authoring area selected in the left Edit Flow navigation.



- `apps/web/editor.html` marks navigation links with `data-editor-area` and editable sections with `data-editor-areas`.
- `apps/web/src/editor.ts` tracks `activeEditorArea`, supports direct URL hashes, and calls `renderActiveEditorArea()`.
- `apps/web/styles.css` hides inactive editor cards and displays only the active workspace's relevant panels.

Milestone 17 Trigger Creation And Linking

Milestone 17 turns trigger editing into trigger authoring. It adds record-level operations and a spatial attachment workflow so rules feel like things placed in the world.

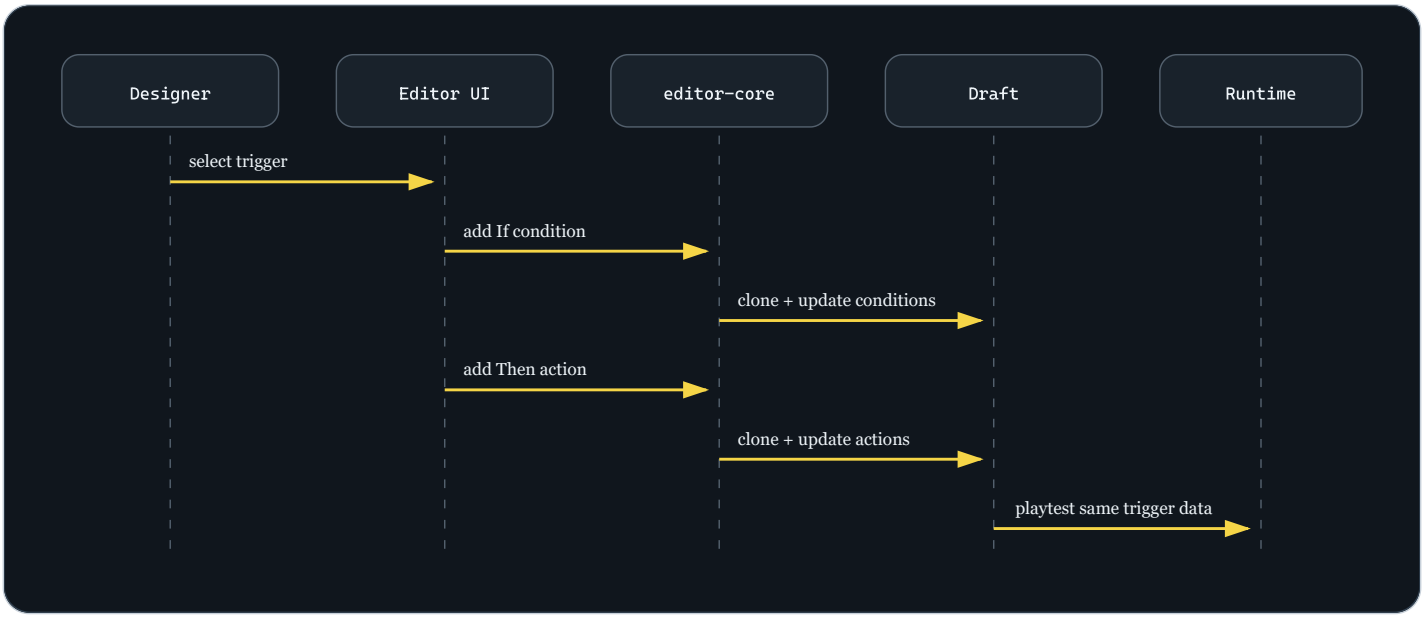


New editor-core operations are `createTriggerDefinition`, `duplicateTriggerDefinition`, and `deleteTriggerDefinition`. The browser editor adds `Create Trigger`, `Duplicate`, and `Delete` controls, plus a `Trigger Markers` Map Workspace mode.

Cells with attached triggers show a small marker chip. The trigger editor also shows a `Referenced Objects` list summarizing linked maps, flags, items, quests, dialogue, teleports, and tile changes.

Milestone 16 No-Code Trigger And Action Builder

Milestone 16 makes trigger authoring feel more like a construction set and less like data surgery. The underlying model did not change: triggers still store `conditions: Condition[]` and `actions: Action[]`, and `runtime-core` still evaluates those arrays through `conditionsMatch(...)` and `applyTriggerActions(...)`. The editor now gives designers guided controls for adding and removing those condition/action objects.



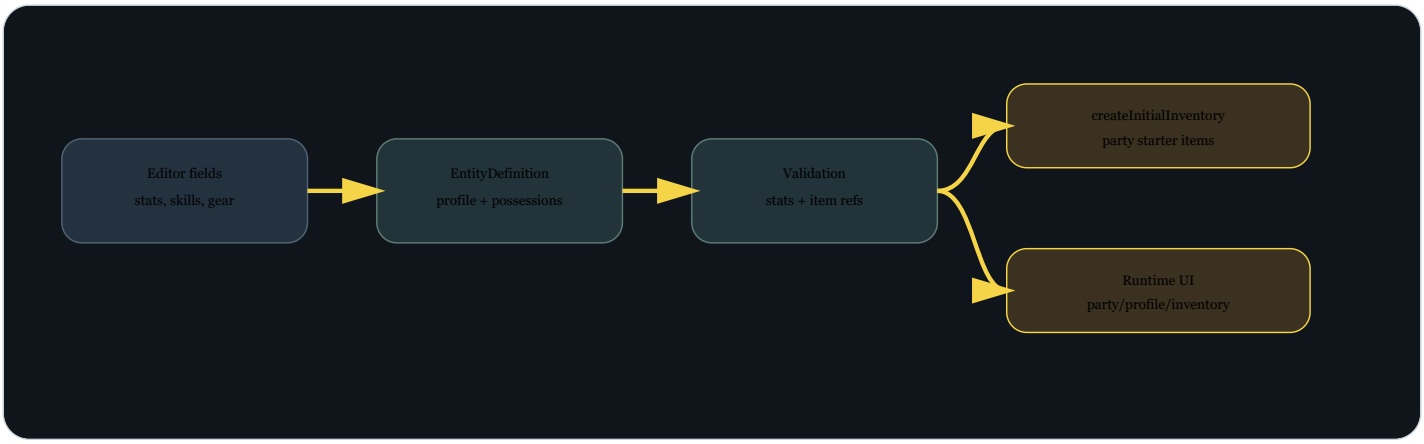
The builder supports `flagEquals`, `hasItem`, and `questStageAtLeast` conditions, plus `showDialogue`, `setFlag`, `giveItem`, `teleport`, and `changeTile` actions.

End to end, a shrine reward can require `quest_started`, show shrine dialogue, grant the Solar Seal, advance `quest_stage`, and change the altar tile to `altar-lit`. The same builder can also express a sci-fi transporter, an urban keypad/locked-door behavior, or a spy dossier pickup without changing the engine.

Historical note: Milestone 16 edited existing trigger records only. Milestone 17 adds trigger record creation, duplication, deletion, and map-cell marker placement; brand-new dialogue/item/quest creation still remains future work.

Milestone 15 Entity Profiles And Starting Gear

Milestone 15 enriches reusable `EntityDefinition` records. Definitions can now carry a `profile` with stat fields and skills, plus `startingPossessions` that seed the party inventory when a new runtime session starts.

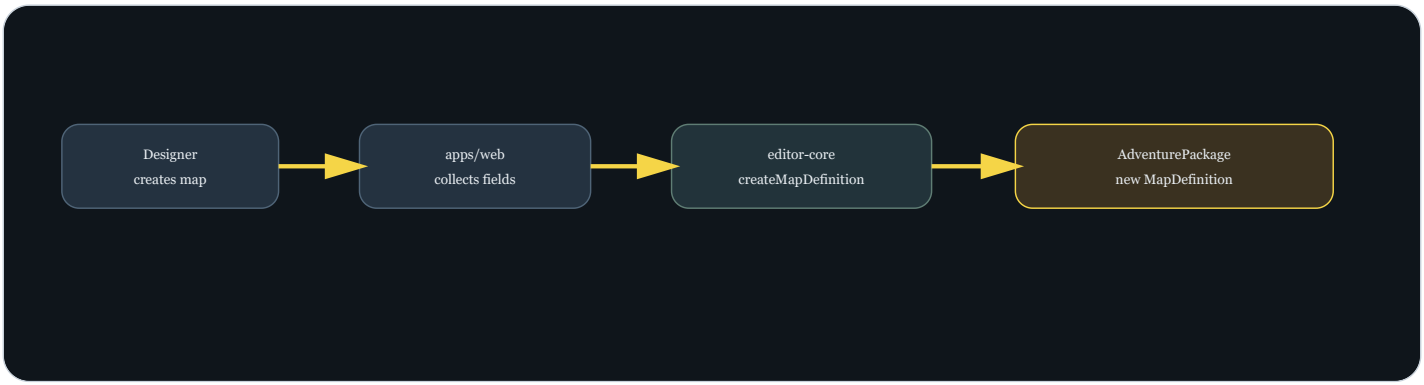


1. The browser editor reads definition form values from `apps/web/src/editor.ts`, including life, power, speed, skills, and starting possession text such as `item_oracle_charm:1`.
2. Those values are stored on the reusable `EntityDefinition`, not on each placed `EntityInstance`. Placement still remains map-specific.
3. `packages/validation` rejects negative/non-integer stats, unknown starting item ids, and invalid starting possession quantities.
4. When `runtime-core` creates a new session, `createInitialInventory` walks the party definition ids in `startState.party`, finds each definition, and folds its `startingPossessions` into inventory state.
5. The runtime browser UI displays the party, profile summary, and named inventory, while `runtime-2d` continues to render from state without owning these rules.

This keeps the future path open: combat can read stats, AI can read skills or faction, equipment can read starting possessions, and alternate renderers can present the same data differently.

Milestone 14 Map Structure Editing

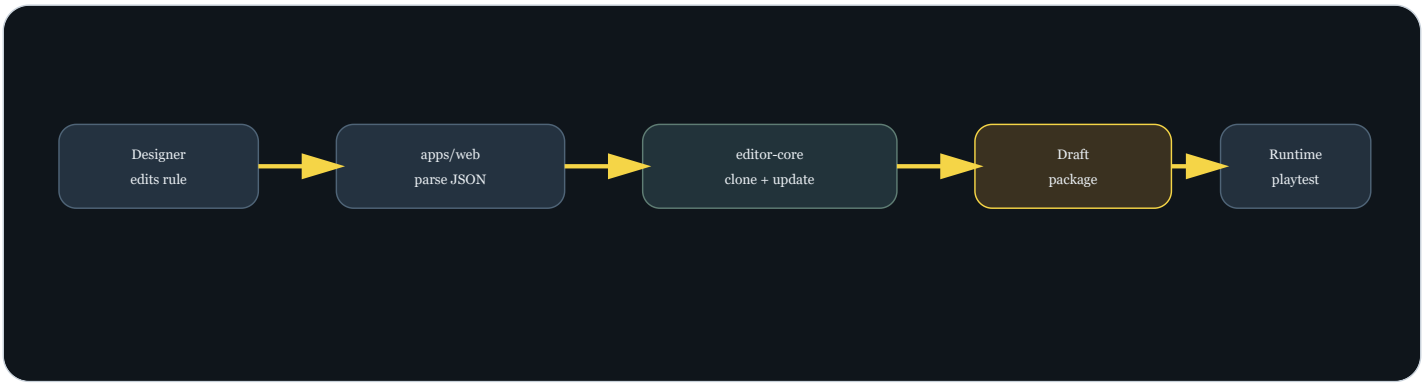
Milestone 14 begins the world-structure track. Maps now have optional category metadata: `world`, `region`, `local`, `interior`, and `dungeonFloor`. The editor can also create a new blank map with a base tile layer.



New maps start with one base tile layer, a repeated fill tile, no exits, and no placed entities. Exit/portal wiring and map deletion remain later milestones because they require careful reference-safety rules.

Milestone 13 Dialogue And Trigger Editing

Milestone 13 adds construction-set controls for existing dialogue records and structured trigger records. Browser controls gather intent, `editor-core` clones and updates the `AdventurePackage`, validation reruns, and playtest loads the same draft package.



Flow: Designer edits dialogue or trigger fields -> `apps/web/src/editor.ts` parses form values and JSON arrays -> `editor-core` updates the package copy -> validation reruns -> playtest loads the same draft.

Dialogue editing updates the first node speaker, text, and continue label. Trigger editing updates type, optional map/x/y location, run-once behavior, conditions JSON, and actions JSON. Invalid JSON is rejected by the editor before it can replace the trigger data.

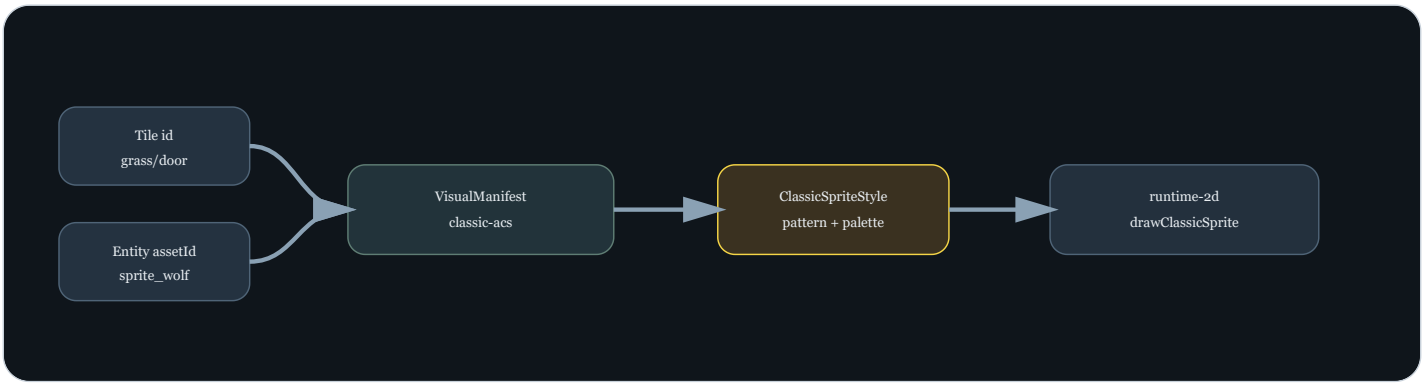
Current limitation: Milestone 13 edits existing dialogue and trigger records. Brand-new record creation, deletion, and a fully visual no-JSON rule builder remain future work.

Milestone 12 Entity Definition Editing

Milestone 12 added browser controls for reusable entity definition metadata. Existing placed entities remain separate `EntityInstance` records that point back to reusable `EntityDefinition` records by id.

Milestone 11 Sprite Manifests

Milestone 11 moves the classic visual mode from hardcoded tile and entity assumptions toward manifest-driven presentation data. `runtime-core` remains renderer-agnostic; `runtime-2d` reads the adventure's `classic-acs` visual manifest when deciding how to draw canvas sprites.

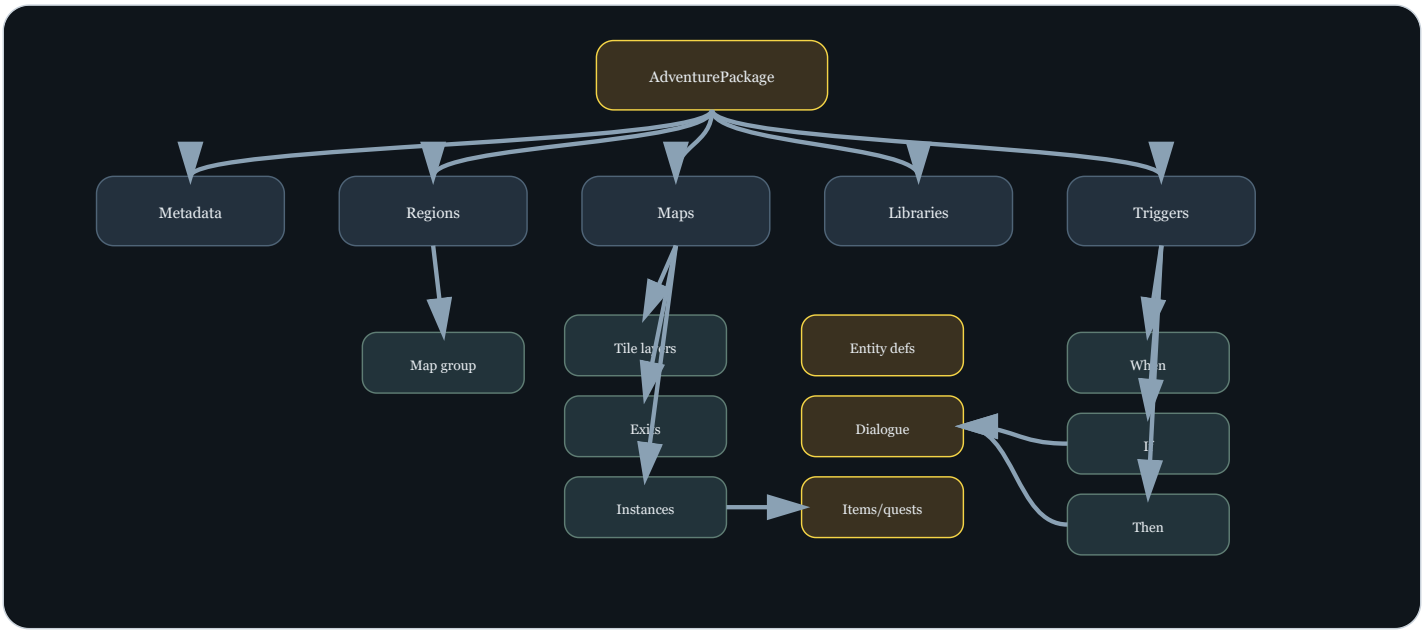


- `AdventurePackage.visualManifests` stores renderer-facing presentation metadata outside the simulation rules.
- The sample adventure defines `Classic Solar Seal Sprite Set` with tile entries for grass, path, shrub, stone, floor, altar, door, water, and void.
- The Hero, Oracle, and Shrine Wolf definitions now point at stable sprite IDs through `assetId`.
- The classic renderer resolves a tile/entity sprite from the manifest first, then falls back to built-in defaults for older packages.
- Validation warns when classic sprite references are missing without blocking the project, because fallbacks keep content playable.

Recommended Editor Information Architecture

The current editor works, but its panels are arranged in milestone order rather than authoring order. As the game data grows, the editor should be organized around the relationships in `AdventurePackage`: Adventure owns Regions, Regions group Maps, Maps contain tile layers, exits, placed entities, and local triggers, while reusable libraries provide definitions that maps and triggers reference.

Core principle: show containment first, references second, and implementation details third. A designer should first understand where they are in the adventure, then what objects live there, then what definitions and rules those objects point to.



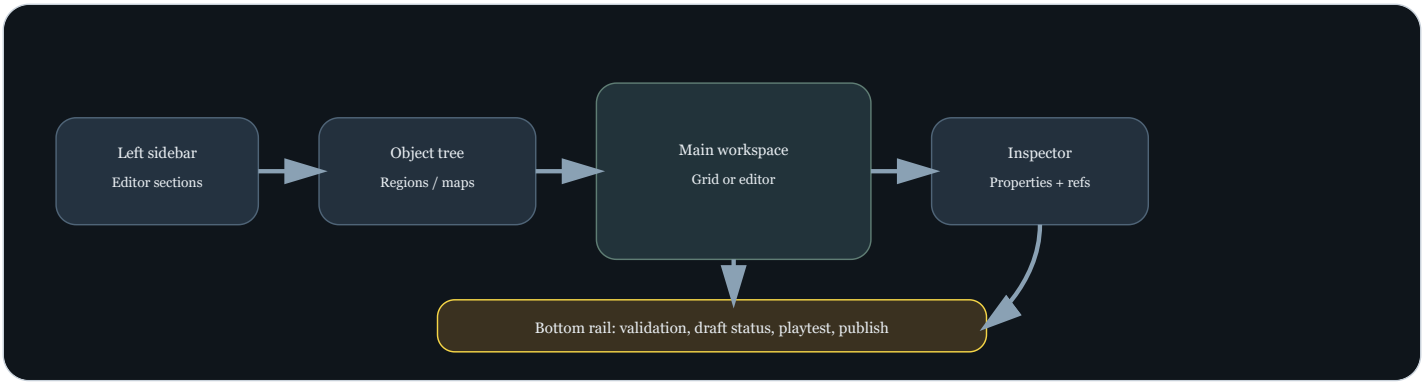
Proposed Editor Areas

Area	Purpose	Primary objects	Why it belongs here
Adventure Setup	Project-level identity and rules	Metadata, rules, start state, validation	These affect the whole adventure and should not be mixed into map editing.
World Atlas	Spatial structure	Regions, maps, map categories, exits, start position	This is the natural home for Milestone 14 map structure tools.
Map Workspace	Editing the selected place	Tile layers, entity instances, local triggers, exits	This is the main canvas-centric construction area.
Libraries	Reusable building blocks	Entity definitions, items, dialogue, quests, assets, visual manifests	These are referenced by maps, triggers, and instances.
Logic & Quests	Cross-map behavior	Triggers, conditions, actions, quest stages, dialogue links	Logic often connects many objects and needs reference-aware tools.
Test & Publish	Workflow actions	Validation, drafts, playtest, projects, releases	These are workflow controls, not content objects.

The editor now uses a simpler ACS-like map hierarchy for designers: `World` for broad overview travel, `Region` for major areas such as districts or station decks, and `Local Area` for the playable scene map. A Local Area can still be a room, cave, street, forest clearing, ship bay, rooftop, office, shrine, or outdoor site.

`Parent Region` is organizational metadata. It tells the World Atlas where a map belongs, but it does not restrict movement. Exits can connect local areas inside one region, local areas in different regions, region maps, or any other valid source/target pair. The domain model can still accept older granular map kinds for compatibility, but the editor-facing flow should prefer the three-part hierarchy unless a future feature proves that more categories are necessary.

Recommended Editor Layout



1. The left sidebar chooses the authoring area: Adventure, World Atlas, Map Workspace, Libraries, Logic, or Test & Publish.
2. The middle column shows the selected area's object tree, such as Regions with nested Maps.
3. The main workspace shows the selected map grid, definition editor, or trigger flow.
4. The right inspector shows properties and references for the selected object.
5. The bottom rail always shows validation status, unsaved draft status, and playtest/publish actions.

Object-Specific Flows

World Atlas

Shows Regions first, then Maps inside each Region. Selecting a map exposes category, dimensions, region assignment, exits, incoming references, and whether the start state points there.

Map Workspace

Centers on a selected map with tabs for Terrain, Entities, Exits, Triggers, and Preview. Selecting a cell should reveal tile id, occupant, exit, and triggers attached to that coordinate.

Libraries

Holds reusable definitions: entity definitions, item definitions, dialogue, quests, assets, and visual manifests. Each library item should show where it is used.

Display-name reskinning should be an editor workflow, not a script or code task. The tool should let designers search across player-facing names and labels, preview affected objects, choose scope, apply changes to the draft, and rerun validation. It should update display fields such as names, titles, speaker labels, descriptions, and selected player-facing text while preserving stable internal ids by default.

The safe default is display-name rename. Advanced internal id refactoring should be a separate, clearly marked workflow because it must update every structured reference and may affect saves, releases, manifests, triggers, quests, and validation.

Logic & Quests

Turns triggers into a `When / If / Then` builder. This can later replace JSON-first editing while still emitting the same domain `TriggerDefinition` data.

Dependency Panel Rule

Every selected object should have a dependency panel. This is the most important UI principle for making relationships understandable.

Selected object	Dependency panel should show
Region	Maps inside the region and any region-level notes/source references.
Map	Region, exits, entities, local triggers, incoming links, and start-state usage.
Map cell	Tile id, occupant, exit, and triggers attached to that coordinate.
Entity definition	All placed instances using that definition and any asset reference.
Entity instance	Definition, map location, placement constraints, and local triggers if applicable.
Dialogue	Triggers/actions that call the dialogue record.
Trigger	Every map, dialogue, item, quest, tile, or entity reference used by its conditions/actions.

Implemented Edit Game Layout

The browser editor now implements the first version of this organization without changing the underlying data model. The same editor functions still update `AdventurePackage`, but the screen is arranged by information hierarchy.

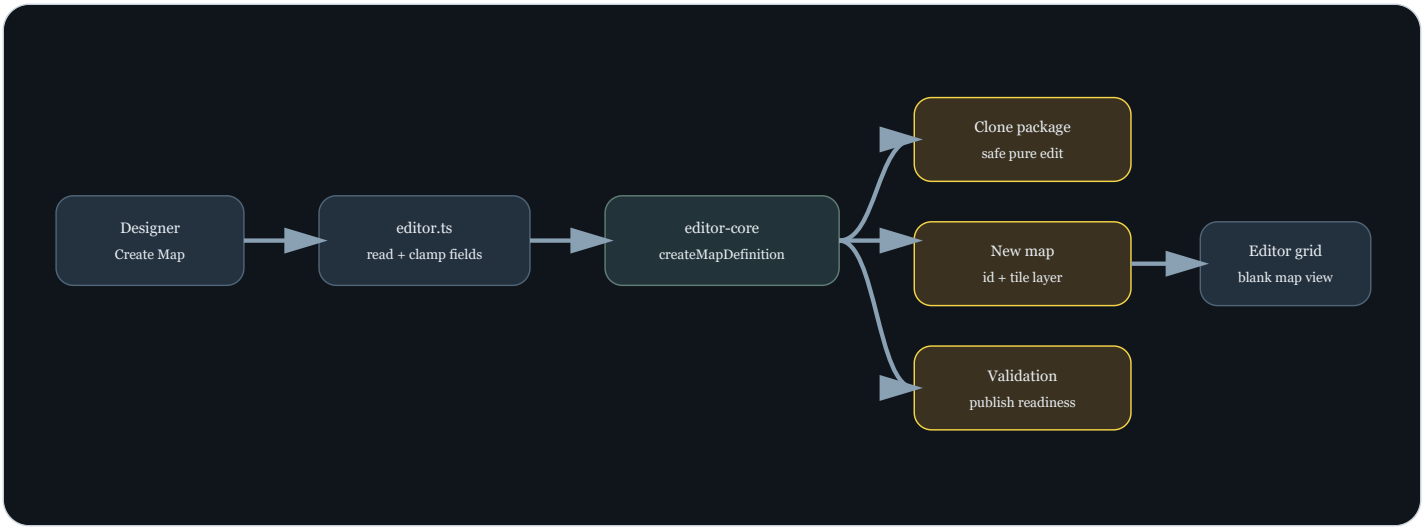
- Left navigation: the authoring flow from Adventure Setup through Test & Publish.
- Context column: Adventure Setup and World Atlas for map selection, region assignment, category metadata, and blank-map creation.
- Center workspace: selected-map editing for terrain and entity instances.
- Inspector column: dependencies, reusable libraries, dialogue, trigger logic, validation, and publishing.

Later milestones can deepen each area with a real region tree, selected-cell inspector, exit graph, reference backlinks, and a structured trigger builder.

Current Feature Implementation Map

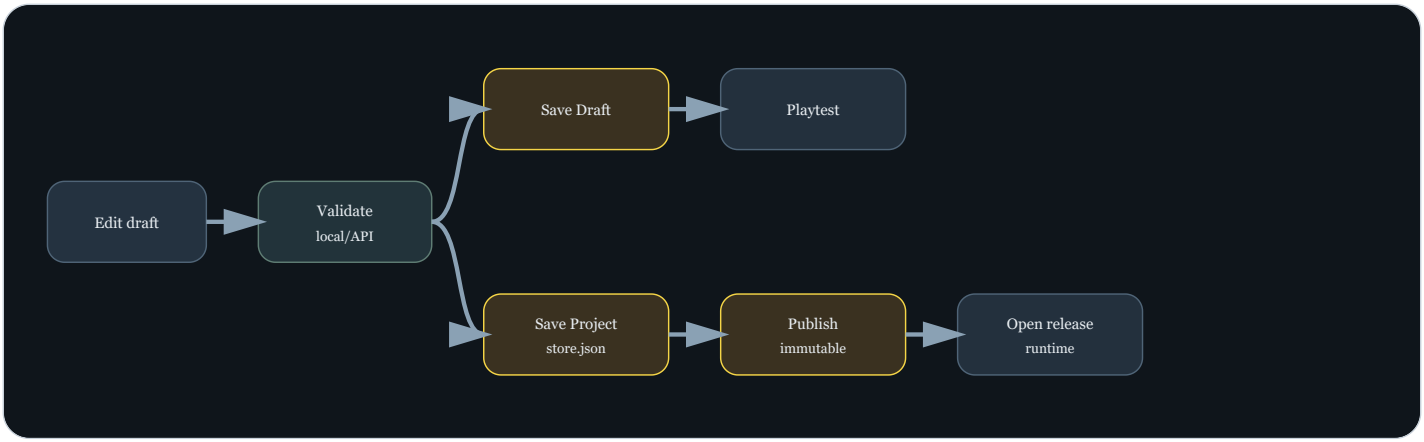
Feature	Browser entry	Core owner	Output
Runtime movement	`apps/web/src/index.ts` keydown	`runtime-core.dispatch` and `handleMove`	State, events, enemy phase, canvas redraw
Inspect/interact/dialogue	`Q`, `E`, dialogue advance keys, Classic scroll arrows	`runtime-core` inspect, interact, dialogue helpers	Event log, Classic message band or Debug dialogue panel, optional state changes
Runtime save/load	`Save` and `Load` buttons	`RuntimeSnapshot` plus `persistence`	IndexedDB snapshot restored into the engine
Visual modes	`Visual Mode` dropdown	`runtime-2d`	Same state rendered as Classic ACS or Debug Grid
Tile brush	Editor grid pointer events	`editor-core.setTileAt`	Map tile layer update, validation, grid refresh
Entity placement/movement	Editor entity mode controls	`editor-core.addEntityInstance` and `moveEntityInstance`	Entity instances update; singleton rules enforced
Definition editing	`Entity Definition` panel	`editor-core.updateEntityDefinition`	Reusable definition changes affect placed instances
Dialogue/trigger editing	`Dialogue` Library Focus and `Rule Trigger` panels	`editor-core.updateDialogueNode` and `updateTriggerDefinition`	Runtime playtest uses edited text/rules
Map structure and creation	`World Structure` panel	`editor-core.updateMapDefinition` and `createMapDefinition`	Map metadata edits and new blank maps
Projects/releases	Project panel buttons	`project-api`, `apps/api`, `validation`	Project drafts and immutable releases in `store.json`

Use Case: Create A Milestone 14 Blank Map



1. The designer enters name, category, optional region, dimensions, and fill tile in the `World Structure` panel.
2. `createBlankMapFromEditor()` trims text, clamps dimensions, and calls `createMapDefinition(...)`.
3. `editor-core` clones the draft package, creates a unique map id, creates one base tile layer, fills it, and appends the map.
4. The browser switches `currentMapId` to the created map, reruns validation, and renders the new grid.
5. The map has no exits yet; reference-safe exit/portal wiring is intentionally future work.

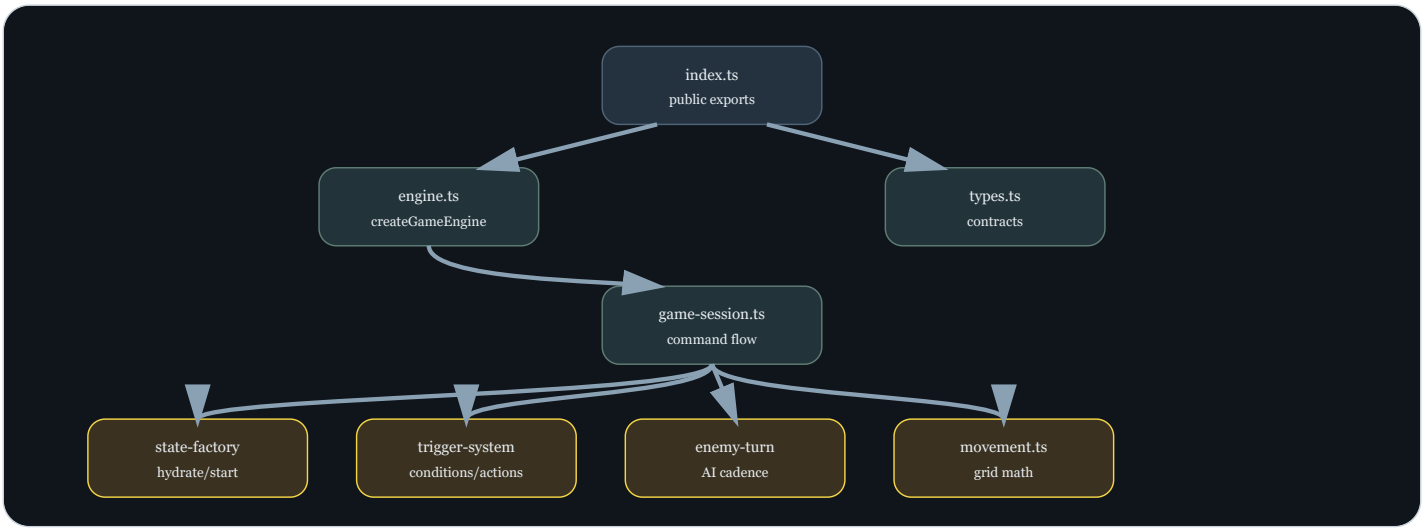
Use Case: Full Authoring Loop



The editor updates an `AdventurePackage`. The runtime, validator, API, and release system all consume that same package shape, which avoids creating a separate editor-only format.

Runtime-Core Internal Module Map

The runtime-core package now keeps `index.ts` as the stable public export surface and moves implementation details into smaller, cohesive modules. That keeps imports stable while making the engine easier to hold in your head.



This split supports the SOLID cleanup path: `RuntimeGameSession` coordinates the command flow, while trigger behavior and enemy behavior now sit behind focused classes. Future cleanup should keep public `index.ts` files as stable export surfaces and move cohesive implementation details into named modules.

Engineering Quality Gates

- `npm run quality:complexity` runs `tools/complexity-check.mjs`.
- New or worsened functions must stay at cyclomatic complexity `8` or below.
- Existing violations are tracked in `tools/complexity-baseline.json` and should be refactored down over time.
- A touched function over complexity `8` should be refactored before or alongside the feature change.
- No new SOLID violations should be introduced: keep responsibilities narrow, prefer registries/handlers for extensible concepts, and keep domain/runtime/editor-core free from browser, API, or persistence details.
- `npm test` is now the milestone completion gate. It runs unit tests, headless editor UI smoke tests, and the runtime playtest smoke script.
- New features should add or update a focused test at the lowest relevant layer. Bug fixes should include regression tests when feasible.

Milestone 29A Test Harness Foundation

The project paused before further feature expansion to add a testing harness. `tools/run-unit-tests.mjs` builds with a verified TypeScript compiler and runs Node's built-in test runner over `tests/unit`. The first unit tests cover runtime-core, editor-core, validation, and persistence. `tools/editor-ui-smoke.ps1` launches the actual browser editor in headless Chromium and verifies progressive disclosure for Terrain, Entity, Exit, and Assets modes. `tools/runtime-ui-e2e.ps1` launches the actual playable browser runtime in headless Chromium and verifies startup rendering, visual-mode and classic-size preference persistence, keyboard movement, interaction, dialogue display, trigger and flag logging, save, reset, and load. `tools/playtest-smoke.mjs` remains the high-level command-runtime acceptance check.

Coverage output is wired through the test harness but disabled by default on the current Node 18 Windows runtime because `NODE_V8_COVERAGE` crashes the process. On a verified runtime, set `ACS_ENABLE_V8_COVERAGE=1` before `npm run test:coverage` to emit V8 coverage JSON.

Runtime browser E2E flow

1. The script starts `apps/web/server.mjs` on an isolated local port.
2. It opens `apps/web/index.html` in headless Chromium through the Chrome DevTools Protocol.
3. It asserts the app version, map name, objective, and canvas startup.
4. It changes Visual Mode and Classic Size, then verifies `localStorage` preference persistence.
5. It dispatches keyboard movement and interaction events, then checks player position, dialogue, event log, trigger firing, and flag summary.
6. It saves, moves, resets, loads, and confirms the saved runtime snapshot restored the player position.

The cleanup strategy is incremental: baseline the current debt, prevent regression, then reduce the baseline during focused refactor passes.

Object Model Corrective Backlog

The rule is not that every visible word becomes an object. Display names, prose descriptions, dialogue text, and lore notes can remain strings. The corrective rule is that any value referenced by runtime rules, validation, editor workflows, starter libraries, AI authoring, or reusable content packs should become a first-class object with a stable id, validation, and editor CRUD.

Area	Current shape	Corrective objective
Quest objectives	`stages: string[]` and numeric stage indexes	Add quest-local objective objects with ids; triggers reference objective ids rather than fragile numbers.
Quest rewards	`rewards: string[]`	Add reward/effect objects or reusable action-backed reward definitions.
Tags	`tags: string[]`	Add managed tag/taxonomy definitions with CRUD and where-used lists.
Flags/state	`flag: string` and runtime `Record<string,...>`	Use `FlagDefId`, typed values, selectors, and save-state migration helpers.
Tiles	Some references still use raw `string` tile ids	Use `TileDefId` consistently and replace manual tile-id inputs with selectors.
Visual sprites	`classicSpriteId?: string` and manifest records	Add sprite/style or asset definition objects with editor CRUD and manifest assignment tools.
Factions	`faction?: string`	Add faction definitions and relationship rules before richer combat.
Dialogue speakers	`speaker?: string`	Reference entity/speaker objects while preserving display overrides.
Dialogue graph	Nodes/choices exist but CRUD is limited	Add full node/choice CRUD, branching, and condition/action-backed choices.
Custom fields	`Record<string, boolean | number | string>`	Add reusable field/schema definitions for custom object classes.

Quest objectives should not become one giant global dropdown. Concrete objectives are quest-local by default, while reusable objective templates/archetypes live in organized libraries. Designers create from templates such as Collect Item, Reach Location, Speak To NPC, Use Item On Tile, Defeat Enemy, Unlock Passage, or Discover Clue, then customize targets, text, rewards, and trigger wiring.

Organization dimension	Examples
Objective kind	progression, interaction, collection, conflict, puzzle, narrative, system
Genre tags	fantasy, sci-fi, modern-spy, superhero, urban-fantasy, supernatural-investigation
Target type	item, entity, map, tile, exit, dialogue, flag, quest objective
Scope	quest-local objective, project template, starter-pack template, imported template
Usage state	recently used, used by current quest, unused, archived

Anti-pollution guardrails: searchable browser instead of infinite dropdowns, where-used counts, duplicate/near-duplicate warnings, reuse/duplicate/create-anyway choices, starter-library namespaces, archive/hide instead of unsafe deletion, and an explicit promote-to-

template flow for quest-local objectives. These tools should appear through focused subpanels, drawers, or temporary warning dialogs, not as permanent screen clutter.

Large TypeScript files should group related interfaces, exported functions, private helpers, sanitizers, and id builders by feature area. Common editors understand collapsible region comments such as `//#region Exits` and `//#endregion`.

```
//#region Exits
export interface UpsertExitInput { ... }
export function listExitsForMap(...) { ... }
export function upsertExitDefinition(...) { ... }
export function deleteExitDefinition(...) { ... }
function createExitId(...) { ... }
//#endregion
```

Cleanup rules: keep interfaces immediately before the functions that use them; group code by feature area; avoid scattering helpers far below their operations; extract large regions into focused modules; and keep the cyclomatic complexity rule intact. This applies to all TypeScript code, including runtime-core, runtime-2d, content-schema, validation, persistence, project-api, apps/web, apps/api, and future packages. `packages/editor-core/src/index.ts` is only the first obvious candidate, not the boundary of the rule.

The corrective object model will add many more reusable objects, but the editor must not show every object and relationship at once. The UI should reveal panels and subpanels only when the current authoring task needs them.

- Keep the top-level Edit Flow focused on one authoring area at a time.
- Use searchable browsers and filter drawers instead of long dropdowns.
- Show duplicate warnings as modal or inline temporary prompts, not permanent panels.
- Place where-used graphs, raw ids, dependency reports, and diagnostics behind expandable details.
- Prefer task-specific creation flows such as `Create Objective From Template` over all-purpose forms with dozens of visible fields.
- Use tabs, accordions, inspectors, and contextual side drawers as panels become richer.

Milestone 23 should start with this corrective pass before creature interaction and combat, because combat needs object-backed factions, drops/rewards, defeat triggers, entity removal rules, and encounter roles.

Milestone 25 Authoring Diagnostics And Playtest Harness

Milestone 25 adds a reusable diagnostics layer beside validation. Validation is the hard package-correctness gate. Diagnostics is the design-quality layer that summarizes authored triggers, entity behavior, exits, flags, inventory objects, quests, and generated playtest scenarios.

Piece	Location	Responsibility
Diagnostics model	`packages/editor-core/src/diagnostics.ts`	Defines diagnostics, scenario, summary, and report records.
Diagnostics builder	`createAuthoringDiagnostics(pkg)`	Inspects the draft and returns compact authoring guidance.
Editor rendering	`apps/web/src/editor.ts`	Runs diagnostics during render and writes diagnostics/scenario rows.
UI surface	`apps/web/editor.html`	Adds Authoring Diagnostics and Playtest Scenarios cards under Test & Publish.
Smoke harness	`tools/playtest-smoke.mjs`	Builds, validates, and simulates key runtime flows.

Diagnostics flow

1. The designer opens `Test & Publish`.
2. The editor reads the current `AdventurePackage` draft.
3. The editor calls `createAuthoringDiagnostics(draft)`.
4. Editor-core inspects triggers, exits, entities, flags, items, and quests.
5. The editor renders compact diagnostics and scenario prompts without cluttering the main workspaces.

Smoke-test flow

1. `npm run playtest:smoke` compiles the workspace.
2. The script normalizes `sampleAdventureData` through content-schema.
3. Validation confirms there are no blocking package errors.
4. Runtime-core receives move/interact commands for the Oracle, shrine reward, and exit travel scenarios.
5. The script prints PASS lines for expected trigger, dialogue, item, tile, quest, and teleport events.

Future Multiplayer Architecture

Multiplayer is planned as a later-phase capability. The recommended foundation is backend-authoritative sessions: browsers submit serializable actor commands, the backend validates them through portable `runtime-core`, advances canonical state, and broadcasts snapshots or event deltas back to clients.

- Keep `runtime-core` DOM-free so solo browser play and server-hosted shared play can reuse the same command validation.
- Use actor-capable action services so players, deterministic NPCs, AI NPCs, and future multiplayer actors all go through the same runtime rules.
- Add actor context to triggers so the engine can tell which player or NPC activated an item, portal, tile, or rule.
- Let persistence store authoritative snapshots and replay/audit events.
- Treat peer-to-peer as optional later exploration, not the initial foundation, because conflict resolution, cheating, and reconnection are harder there.

Future Mobile Play Mode

Mobile support should mean play mode first. Phone-sized devices should not expose editor/create workflows. A future mobile shell should hide or block authoring tools through screen-size and capability gates while reusing the same runtime state and command model.

- Touch controls should dispatch the same move, inspect, interact, continue-dialogue, and profile commands as keyboard play.
- The play UI needs readable scaling, message-band support, current objective/state display, and an inventory/profile drawer.
- Standalone playable publishing should be able to emit a direct play URL or installable play shell without editor source controls.
- Larger tablets can be evaluated later, but phone editing is intentionally out of scope.

Pre-Milestone Build Documents

Accepted recommendations should not live only in chat. Before each milestone build, consult the durable planning trail: `docs/roadmap.html`, `docs/system-reference.md`, `docs/llm-project-context.json`, `docs/codex-chat-log.json`, `docs/user-guide.md`, and `docs/quality/engineering-quality.md`.

When a recommendation is accepted during collaboration, the next documentation pass should mirror it into the relevant durable document before or alongside implementation.

Documentation Generation Requirements

- After every milestone, update the User Guide tutorial so it tries every feature currently available.
- Explicitly highlight the latest milestone's features in the User Guide and System Reference.
- Keep Mermaid diagrams in Markdown and readable rendered equivalents in HTML/PDF.
- Include current runtime/editor screenshots or screenshot-style graphics where they help explain behavior.
- Document each major feature with an end-to-end flow from browser input to core operation to validation, persistence, or rendering output.
- Avoid broken image links, HTML scroll artifacts, overlapping diagram text, and page splits through important diagrams or callout boxes.

UX Skinning Inventory

The live UI surface inventory for future skinning is tracked in `docs/ux-skinning-inventory.md` and `docs/ux-skinning-inventory.json`.

Those files catalog the current Edit Mode and Play Mode elements that exist in the real application, including panels, buttons, text inputs, textareas, dropdowns, multi-selects, checkboxes, lists, grid cells, overlays, canvases, diagnostics cards, publishing/release handoff cards, and dynamic DOM fragments created by the editor and runtime presenters.

This inventory exists because the project intends to support future skins for both the editor and the game player. A later skin should be able to replace borders, backgrounds, icons, workflows, typography, and panel treatments without changing authoring behavior or gameplay rules. Any milestone that changes visible UI is expected to update the inventory so the eventual skinning phase has a current map of the UI surface area.

Extension Points

- A higher-resolution renderer can live beside `runtime-2d` as long as it consumes `AdventurePackage` and `GameSessionState`.
- A future 3D renderer should not change browser input or editor logic unless the content model needs richer spatial metadata.
- Real-time play would most likely evolve the command scheduling, enemy `turnInterval`, and enemy phase in `runtime-core`.
- Richer enemy AI belongs in `runtime-core` or a future AI package, not in the DOM or canvas renderer.
- New editor creation tools should start as pure functions in `editor-core`, then be wired into `apps/web/src/editor.ts`.